Design Document

for

PLACEIIT

Version 1.0

Prepared by Team Soft_warriors

Group 2: **Group Name: Soft_warriors**

Joy Vardhan Yalla	241192	yallaj24@iitk.ac.in
Pravesh Thalor	240791	pravesh24@iitk.ac.in
Sumath Rengasami V	241054	sumathr24@iitk.ac.in
Duvvuri Jyothi Samanvith	240380	jyothids24@iitk.ac.in
Tumati Harsha Vardhan Reddy	241107	tumati24@iitk.ac.in
Krish Mesariya	240560	kmesariya24@iitk.ac.in
Kartik S Basant	240527	kartiksb24@iitk.ac.in
Mehta Devarsh	240650	devarshms24@iitk.ac.in
Abhirup Vasa	241147	vasa24@iitk.ac.in
Anoop Muthavarapu	240672	anoopm24@iitk.ac.in

Course: CS253

Mentor TA: Seetaramayya Lavu

Date: 06 February 2026

CONTENTS II

REVISIONS II

1 CONTEXT DESIGN **4**

1.1 CONTEXT MODEL 4

1.2 HUMAN INTERFACE DESIGN 6

2 ARCHITECTURE DESIGN **23**

3 OBJECT-ORIENTED DESIGN **29**

3.1 USE CASE DIAGRAM 29

3.2 CLASS DIAGRAM 38

3.3 SEQUENCE DIAGRAM 40

3.4 STATE DIAGRAM 57

4 PROJECT PLAN **59**

APPENDIX A - GROUP LOG **62**

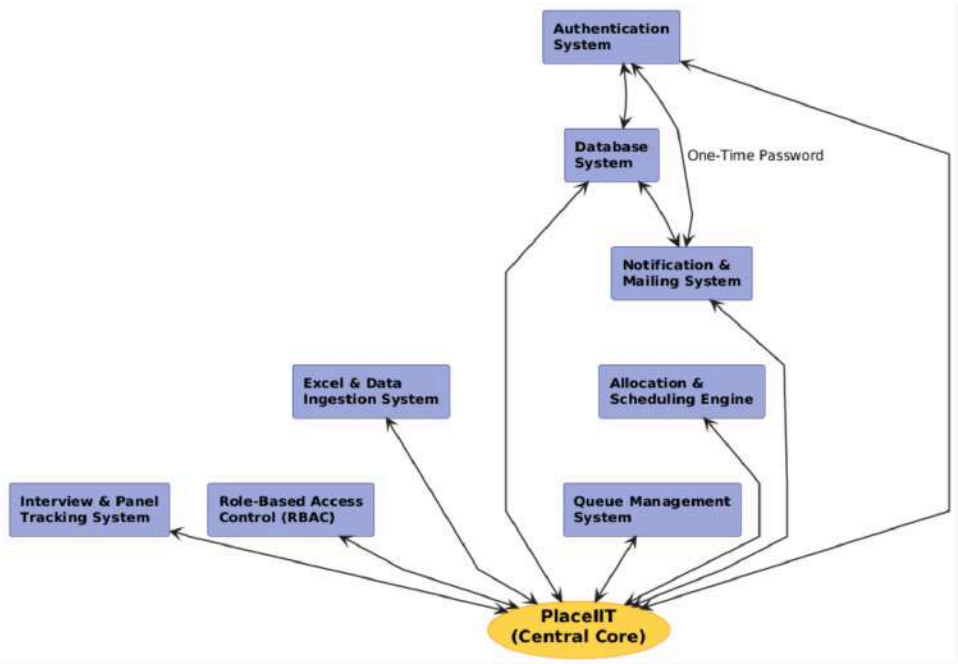
`

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Initial Draft (v1.0)	Abhirup Vasa Kartik S Basant Pravesh Thalor Joy Vardhan Yalla Tumati Harsha Vardhan Reddy Devarsh Mehta Sumath Rengasami V Krish Mesariya Anoop Muthavarapu Duvvuri Jyothi Samanvith	Initiated the document. Added all the necessary details.	06 February 2026

● Context Design

1.1 Context Model

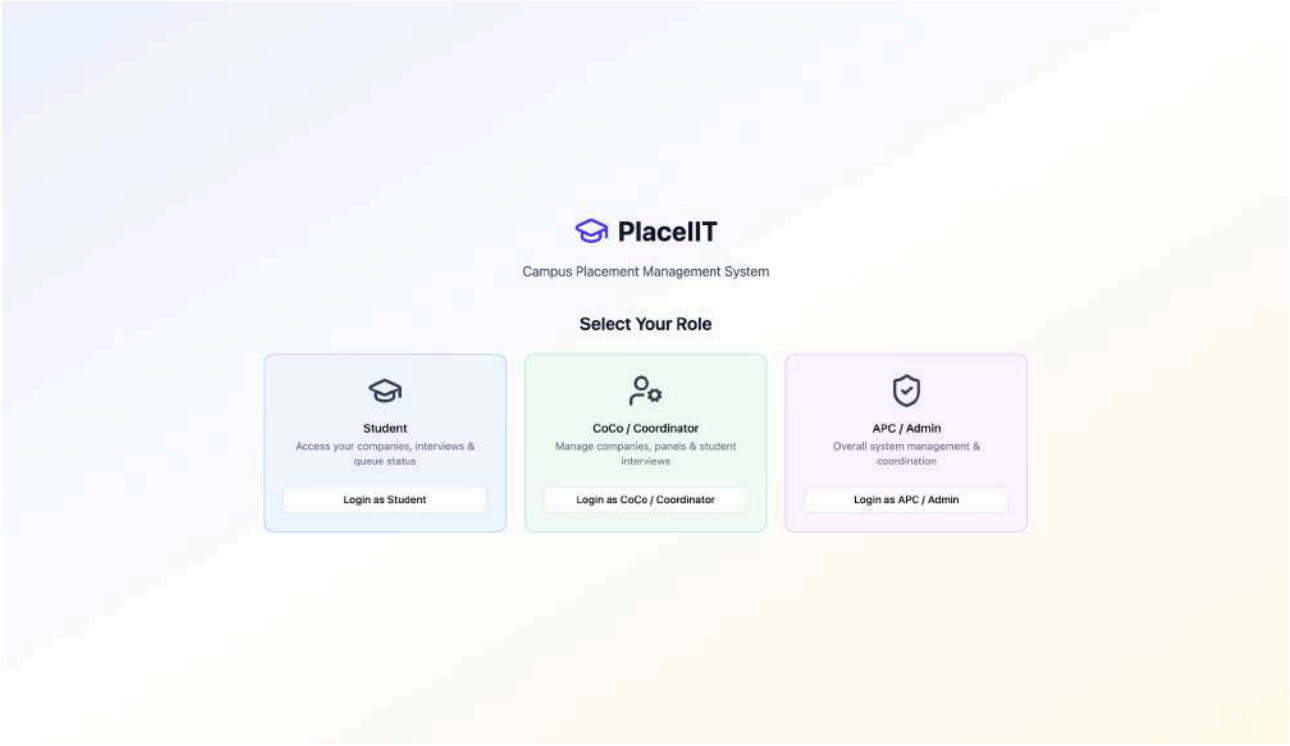


- **Authentication System:** This system securely handles the task of authenticating users based on their pre-selected role (Student, CoCo, or APC) using institute credentials, granting access to the respective dashboard upon successful verification.
- **Mailing System:** The mailing system is exclusively responsible for sending One-Time Passwords (OTPs) for identity verification during first-time login and password reset procedures.
- **Database System:** This system stores all persistent information pertaining to the software, including student profiles, company details, interview schedules, and real-time queue states.
- **Queue Management System:** This system acts as a centralized engine to track live interview queues, calculate real-time positions for students, and manage the "Walk-in" drive listings.
- **Interview & Panel Tracking System:** This system allows Coordinators to create and manage interview rounds, monitor panel status (idle vs. busy), and update student statuses (e.g., "In Interview", "Completed") in real-time.
- **Role-Based Access Control System (RBAC):** This control system assigns users strictly defined levels of access, ensuring Students, Coordinators, and Admins only view data relevant to their specific roles.

- **Excel & Data Ingestion System:** This system handles the parsing and processing of bulk-uploaded Excel files for company details, student shortlists, and coordinator requirements.
- **Allocation & Scheduling Engine:** This system runs algorithms to perform automated, randomized allocation of Coordinators to companies and manages the filtering of companies based on Days and Slots.
- **Notification System:** This system manages the delivery of real-time web-based alerts and platform updates to students regarding queue positions, coordinator messages, and venue changes directly on the website dashboard.

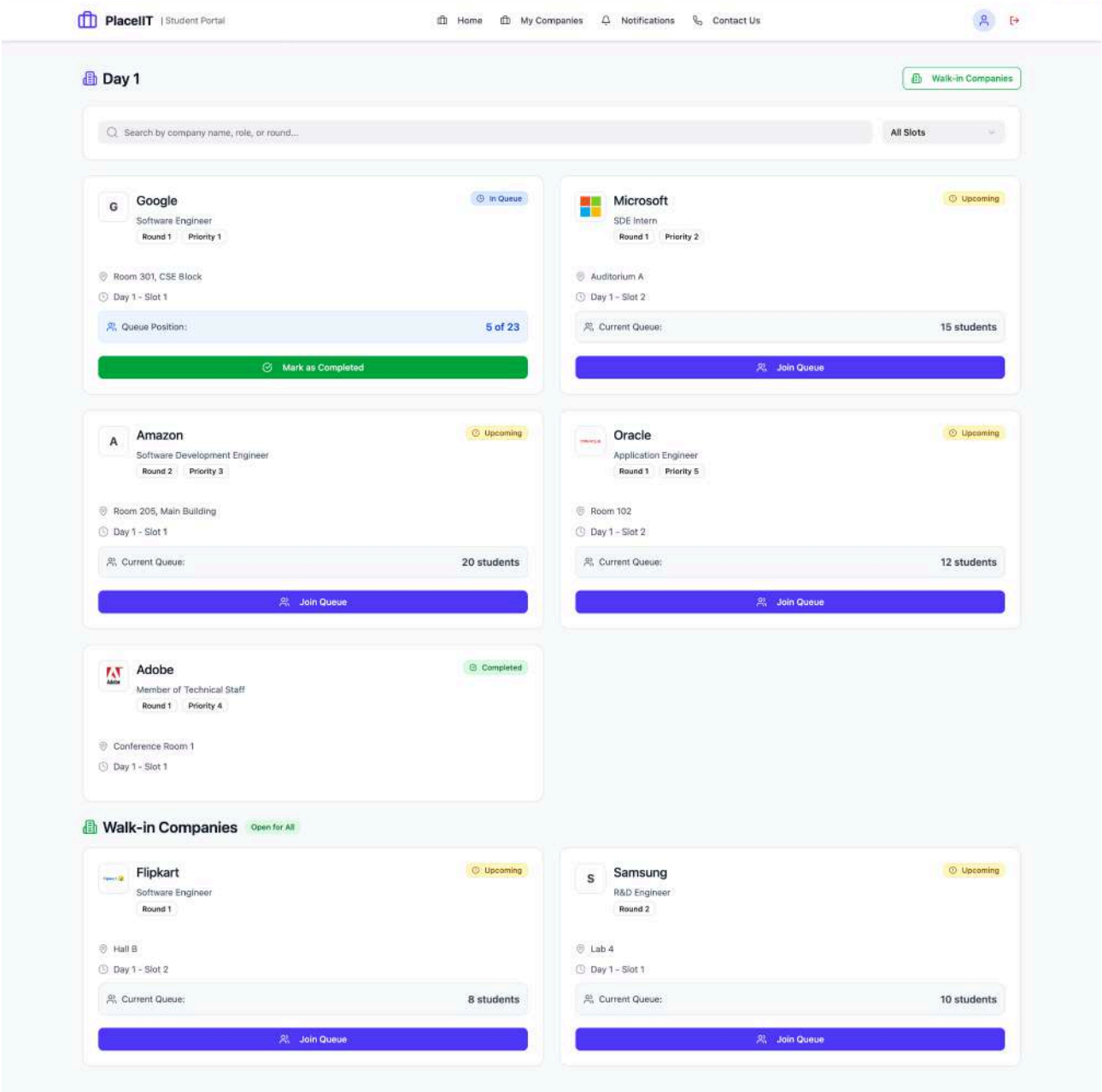
1.2 Human Interface Design

1.2.1 Login

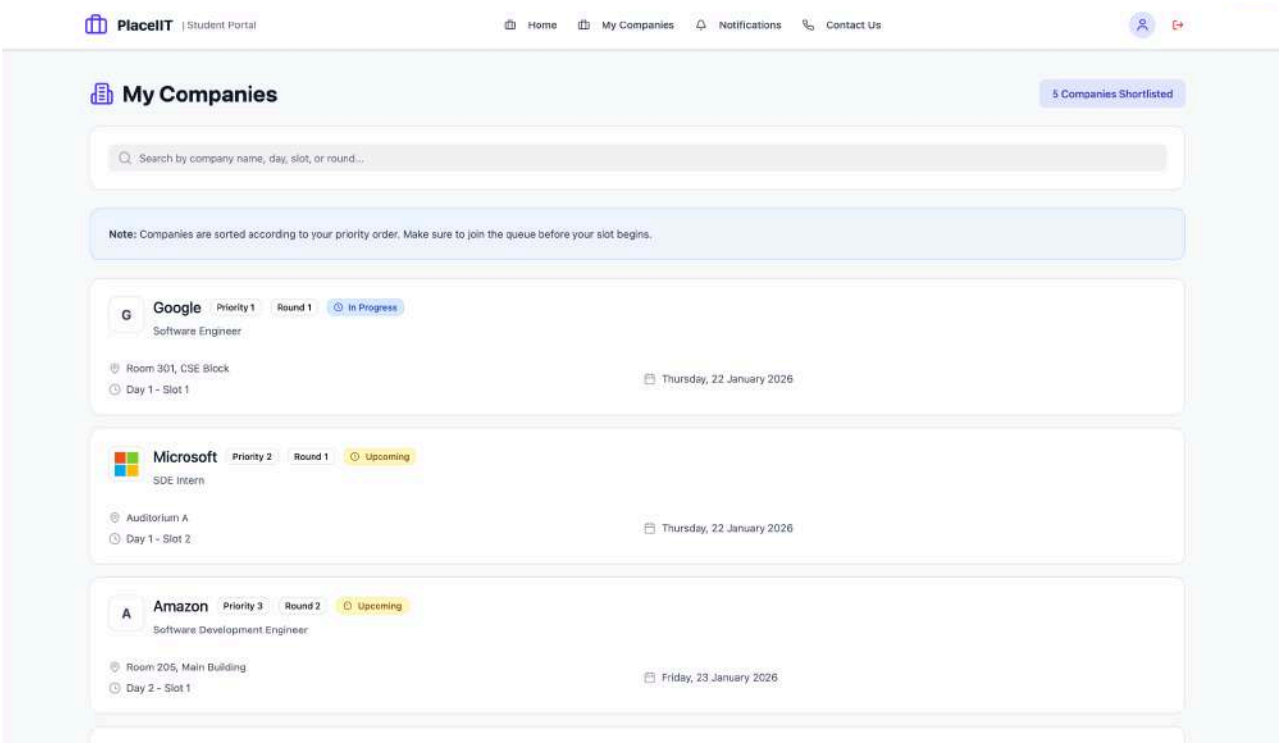


1.2.2 Student Page

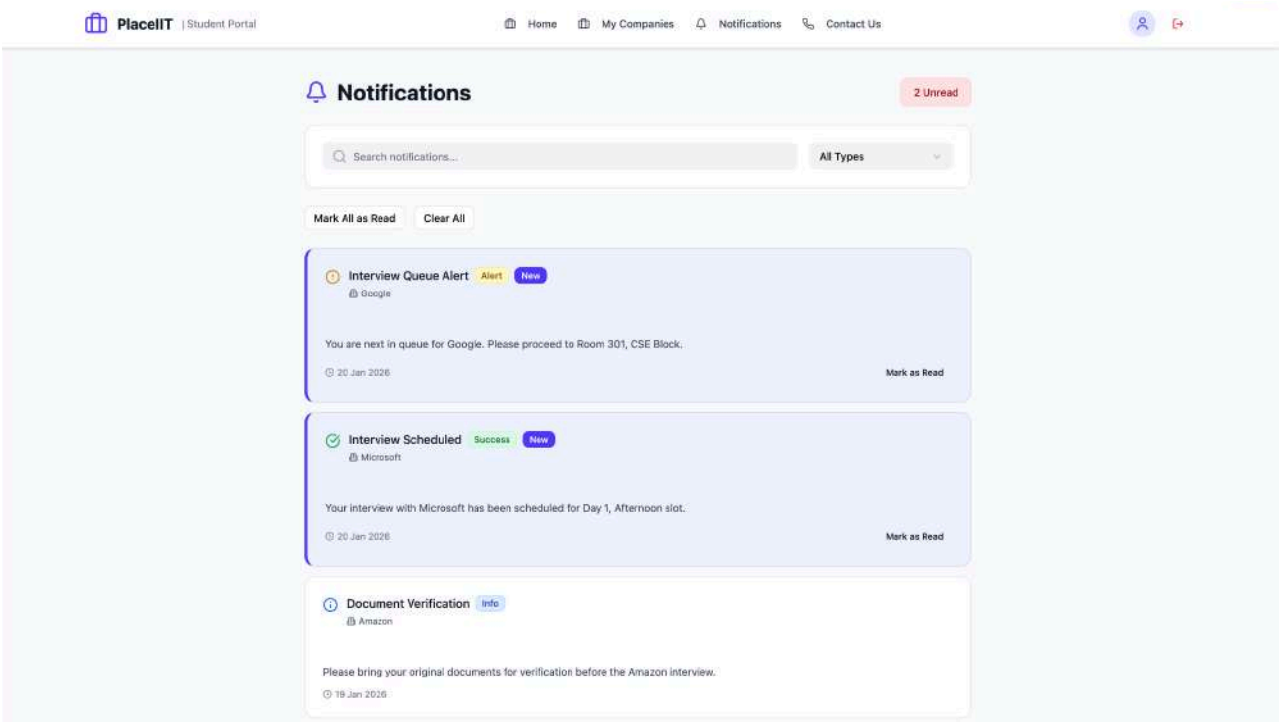
1.2.2.1 Home Page



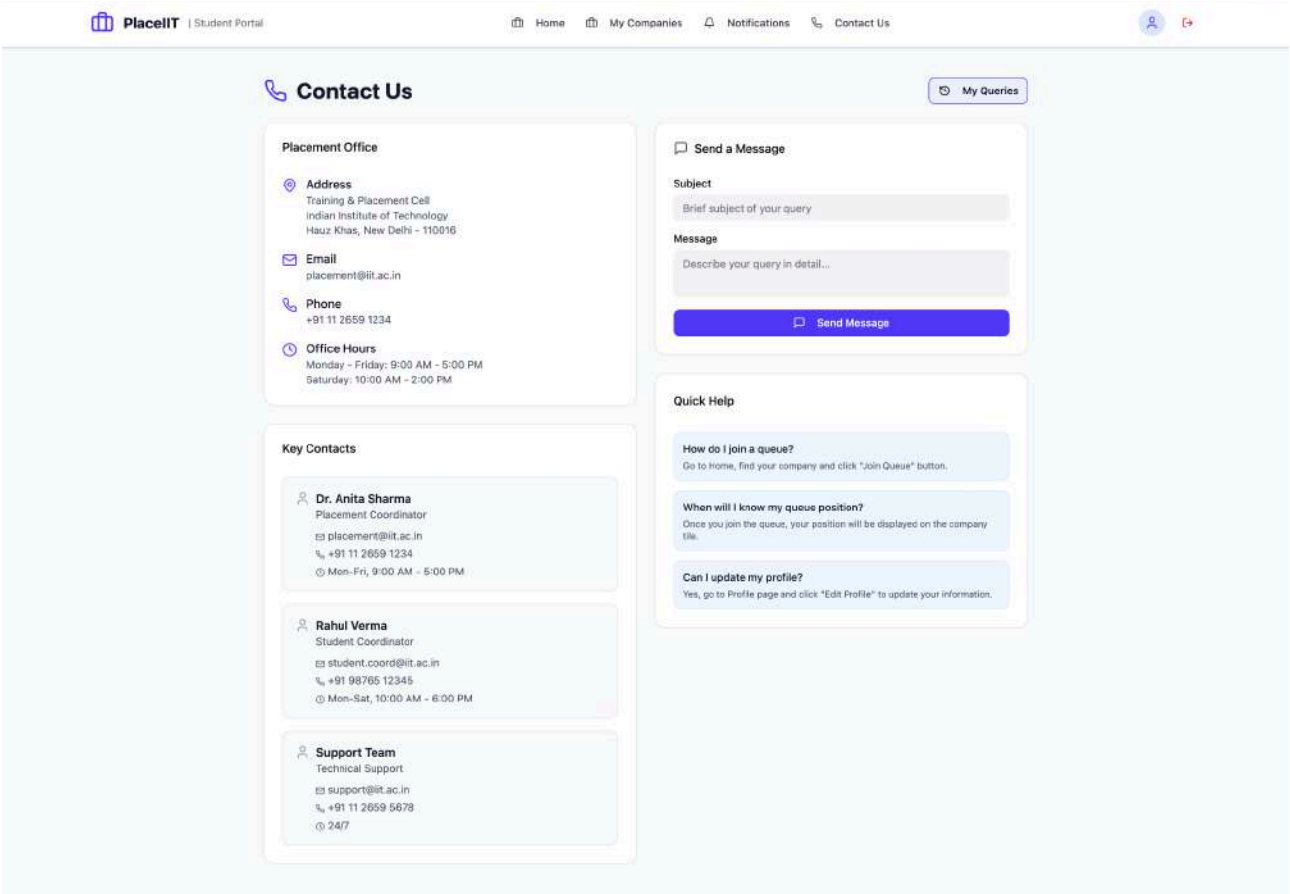
1.2.2.2 My Companies



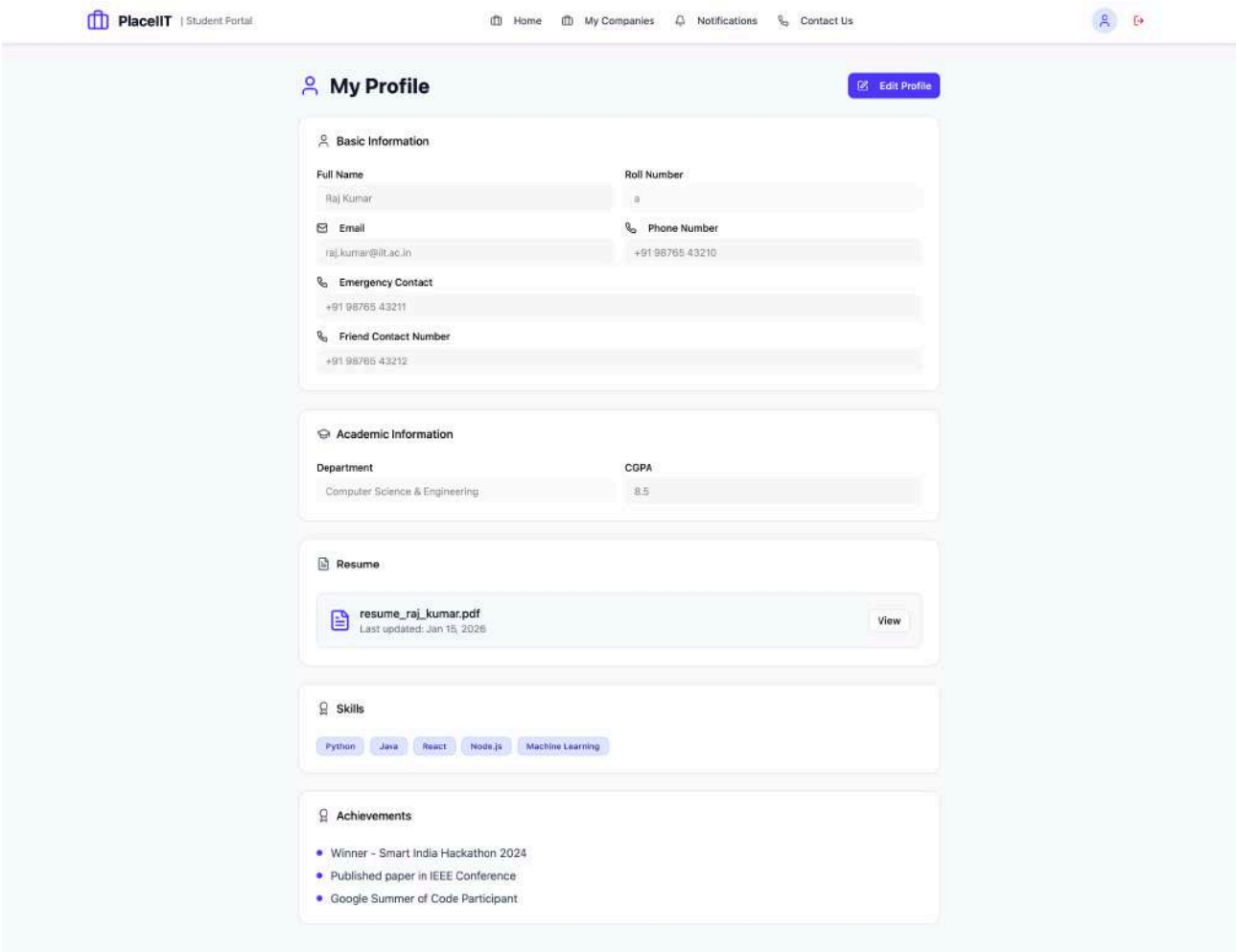
1.2.2.3 Notification



1.2.2.4 Contact Us

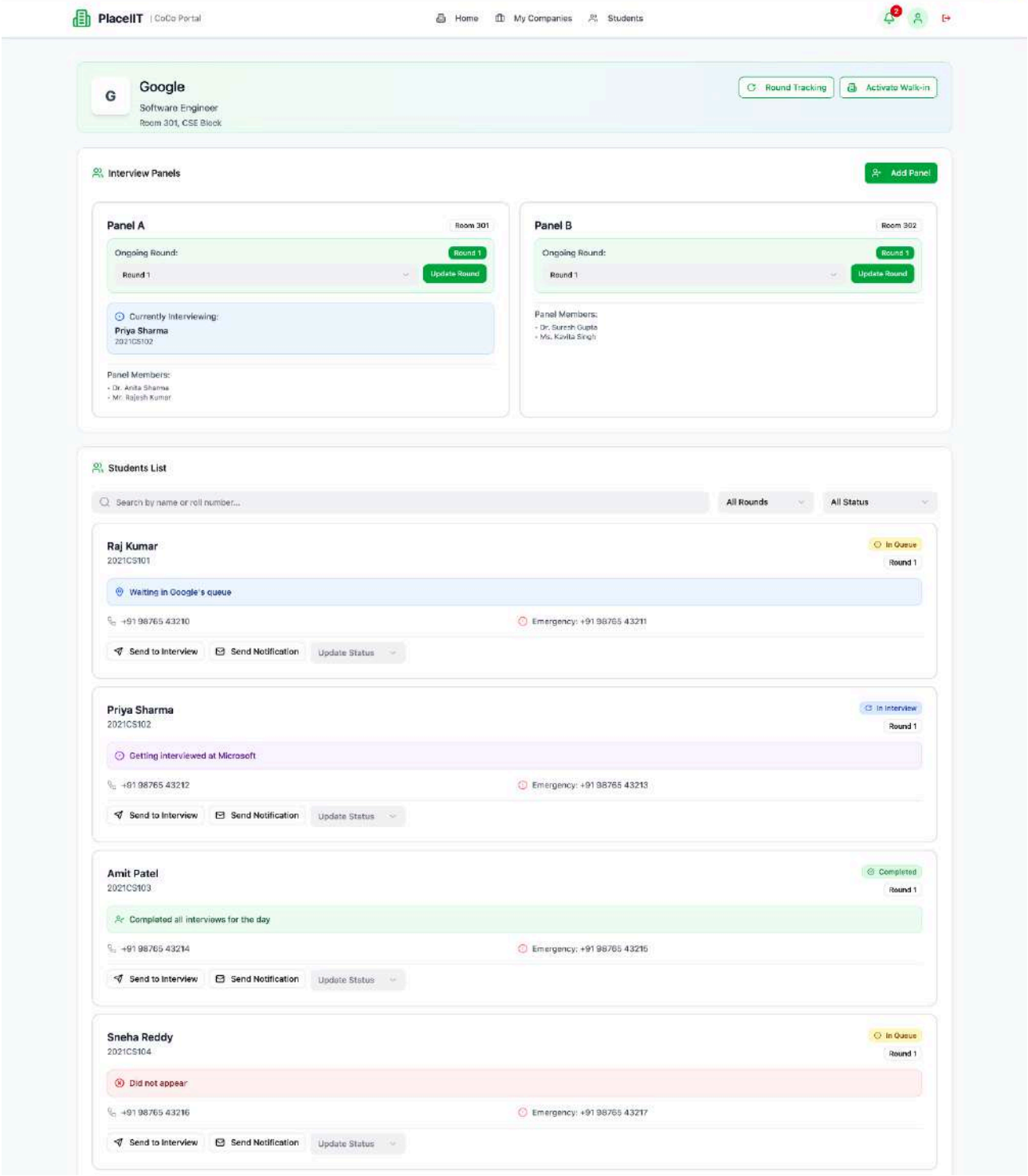


1.2.2.5 Profile Page

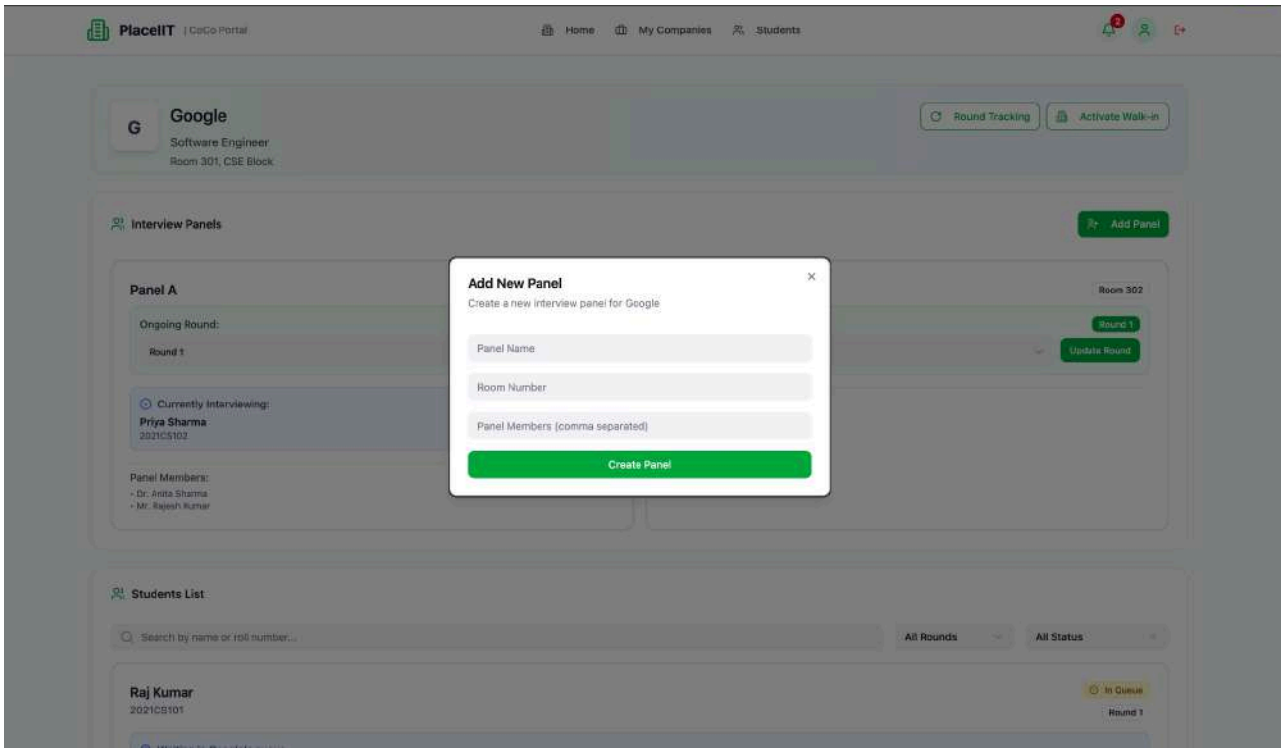


1.2.3 CoCo

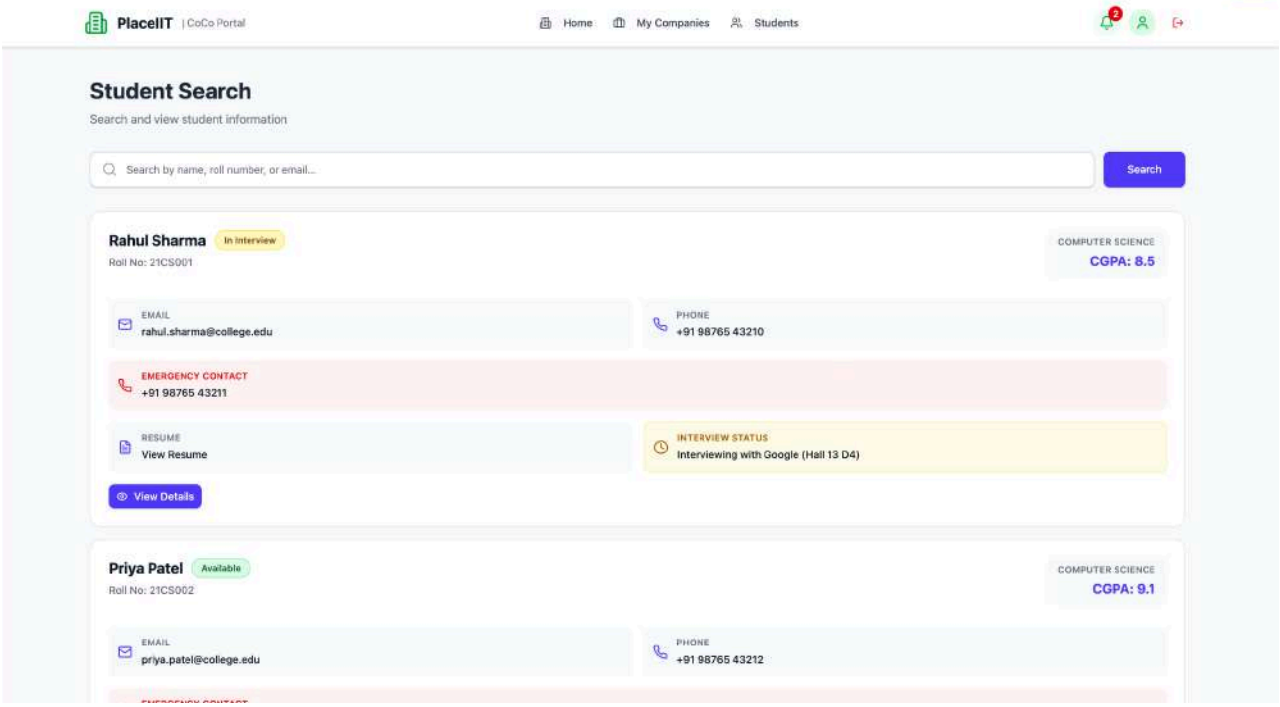
1.2.3.1 Home Page



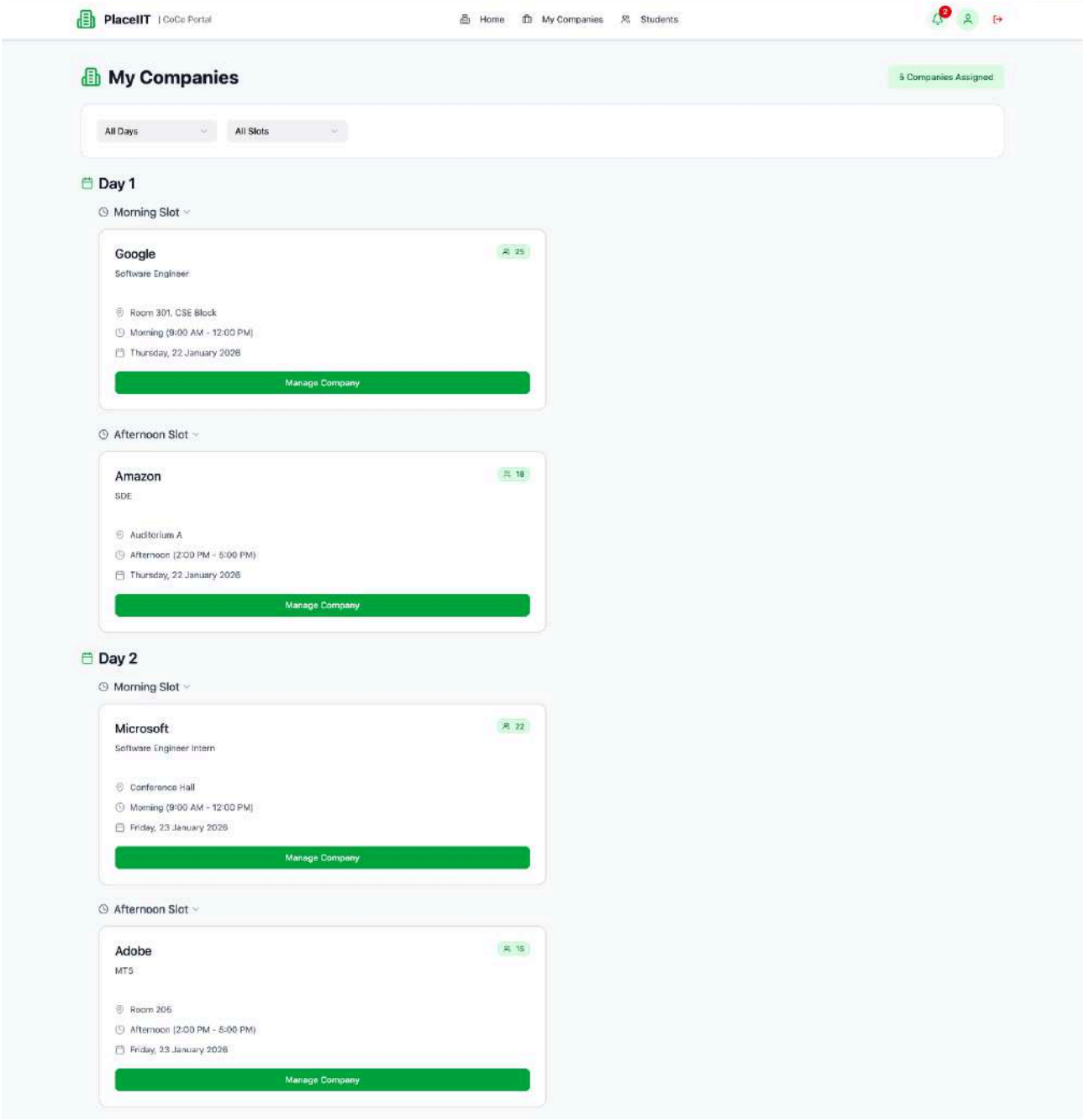
1.2.3.2 Add Panel



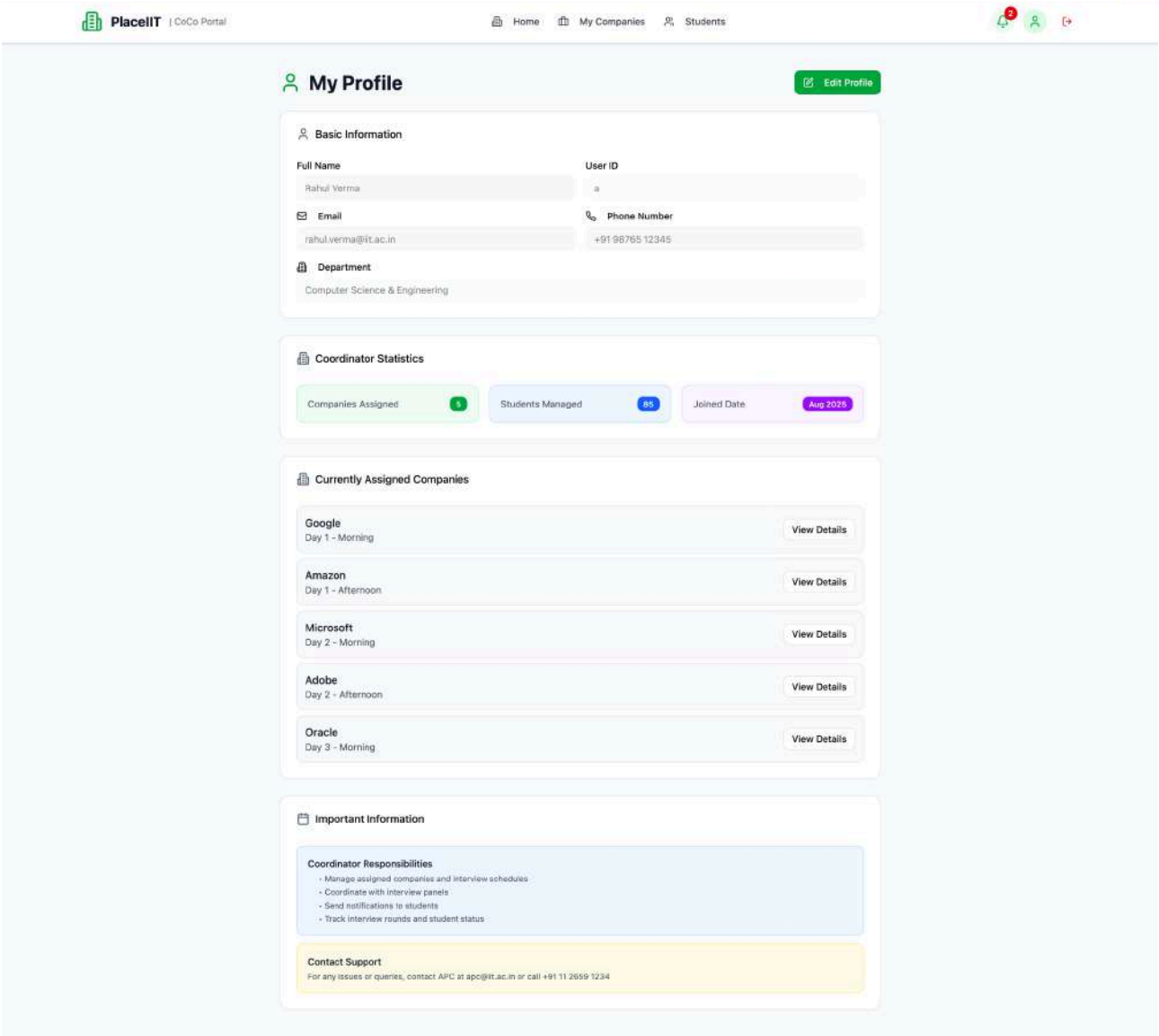
1.2.3.3 Students



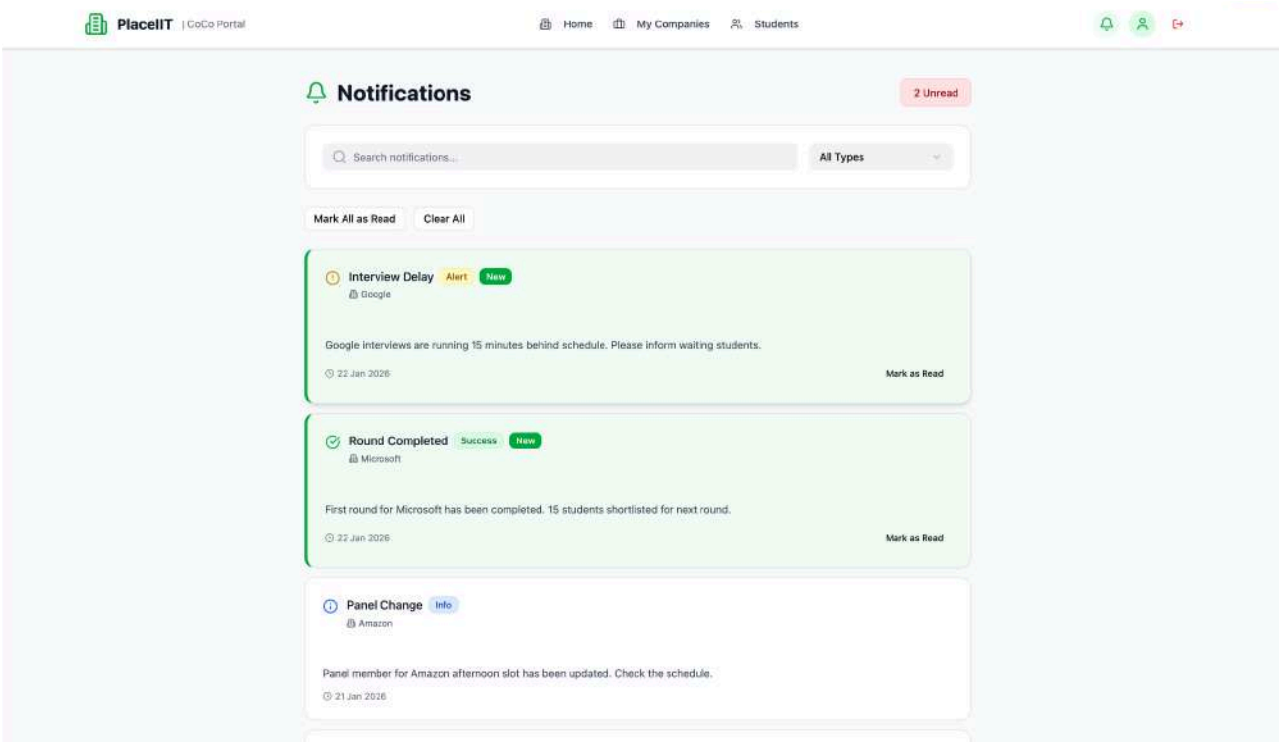
1.2.3.4 My Companies



1.2.3.5 Profile Page

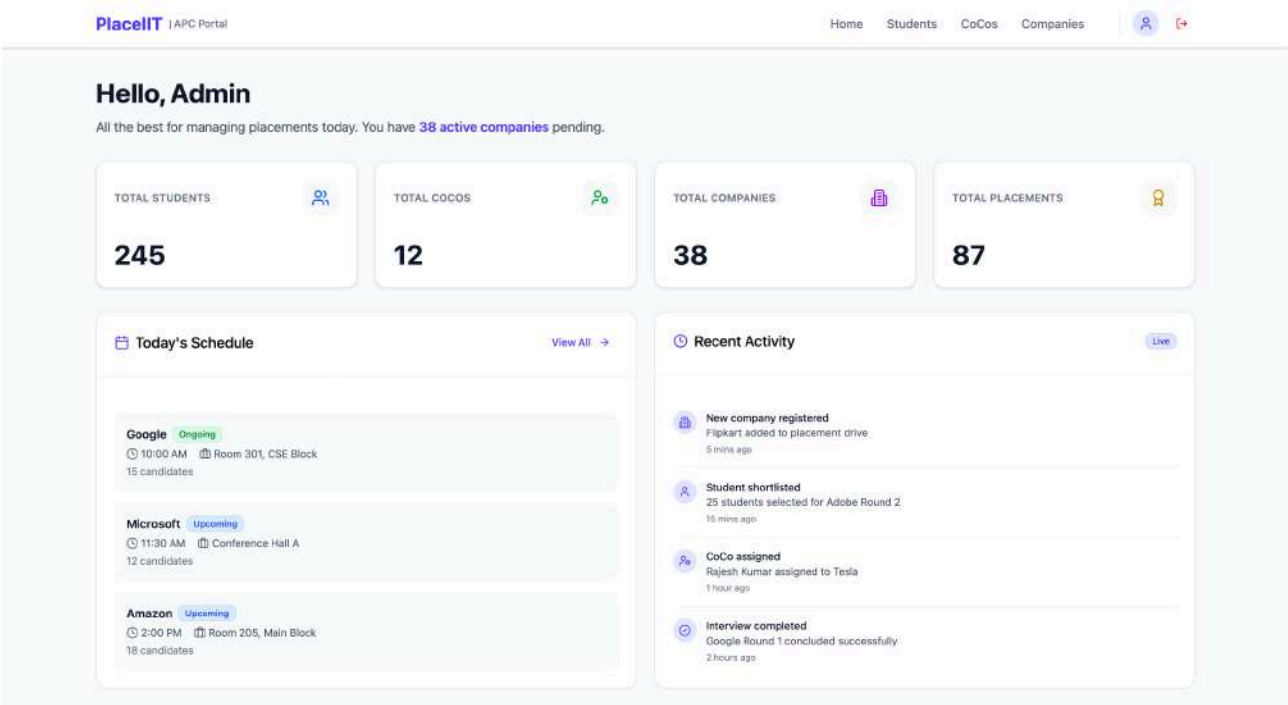


1.2.3.6 Notification

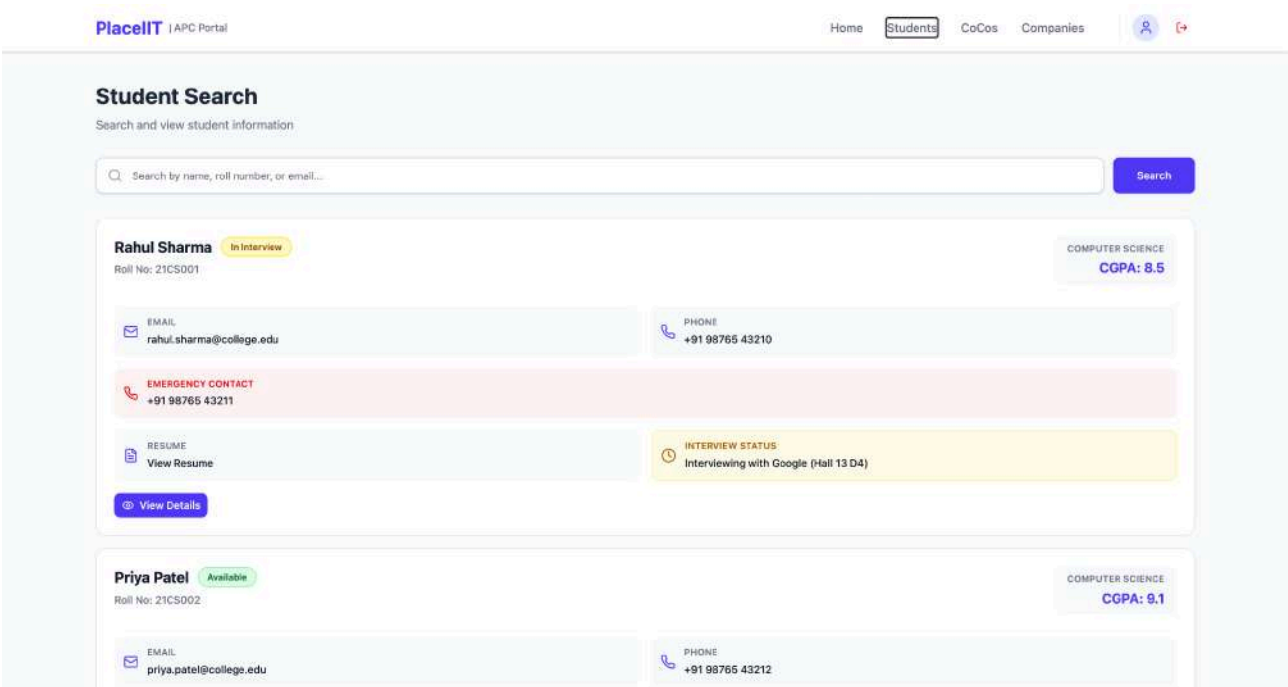


1.2.4 APC

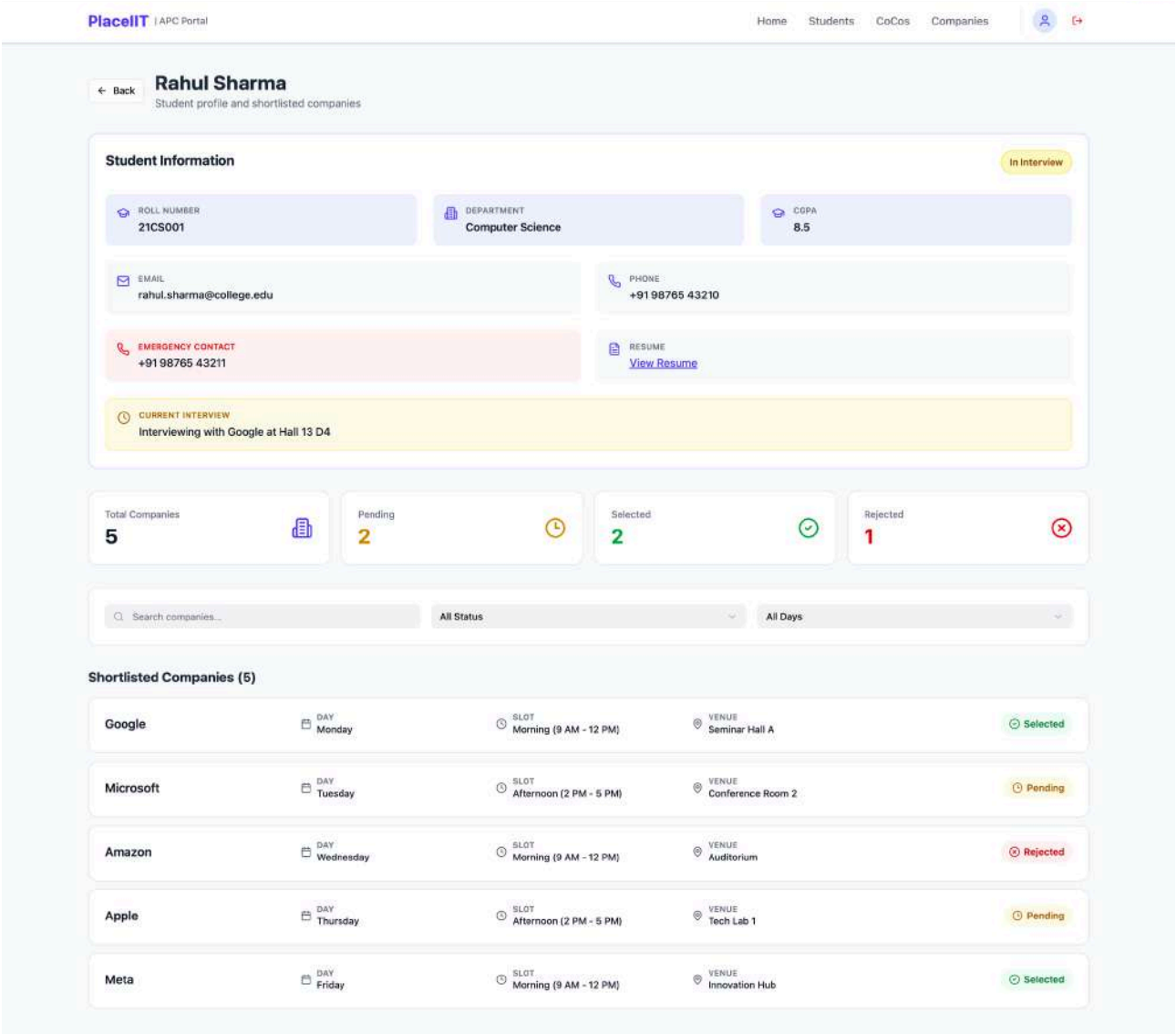
1.2.4.1 Home



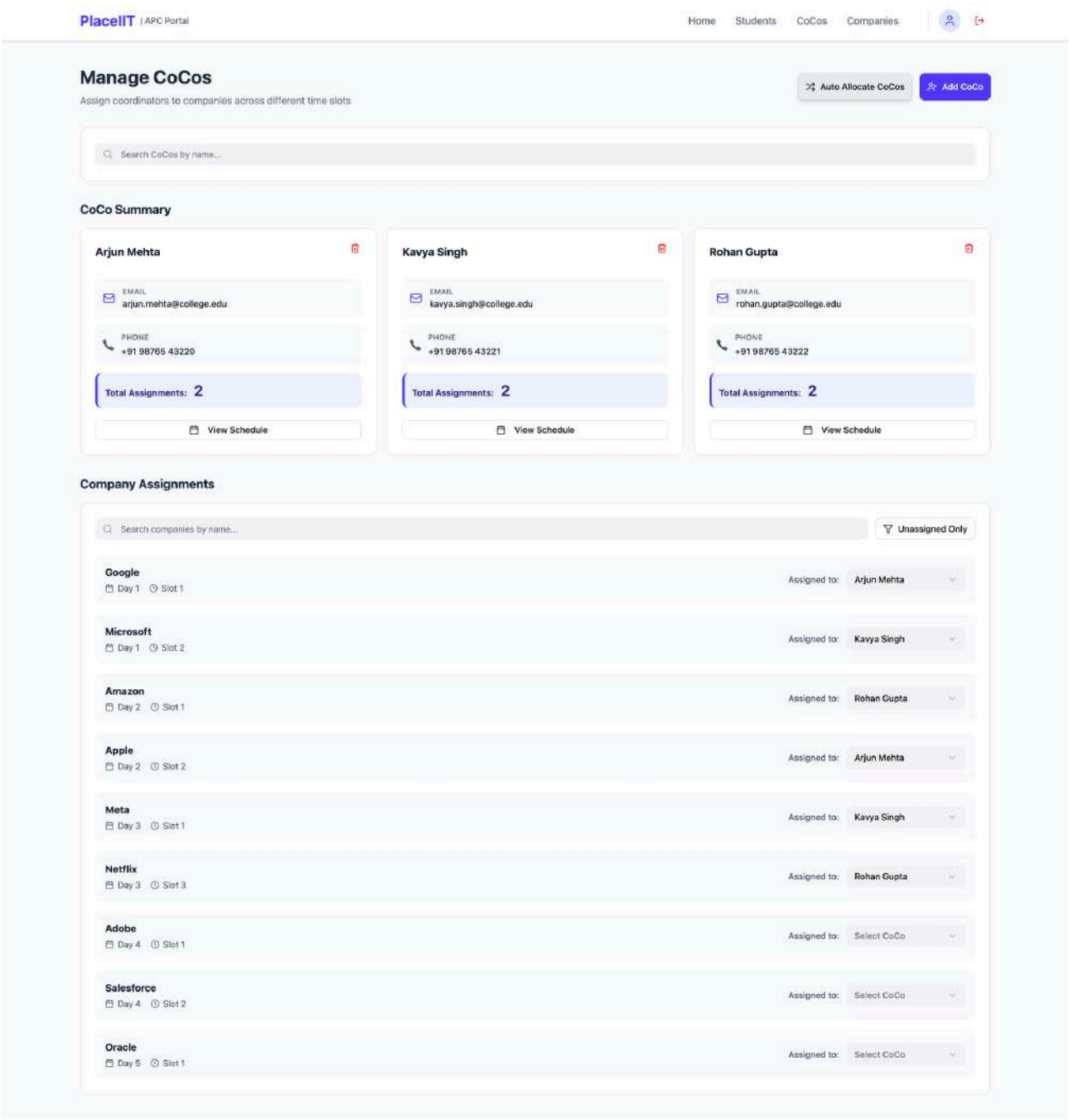
1.2.4.2 Student Search



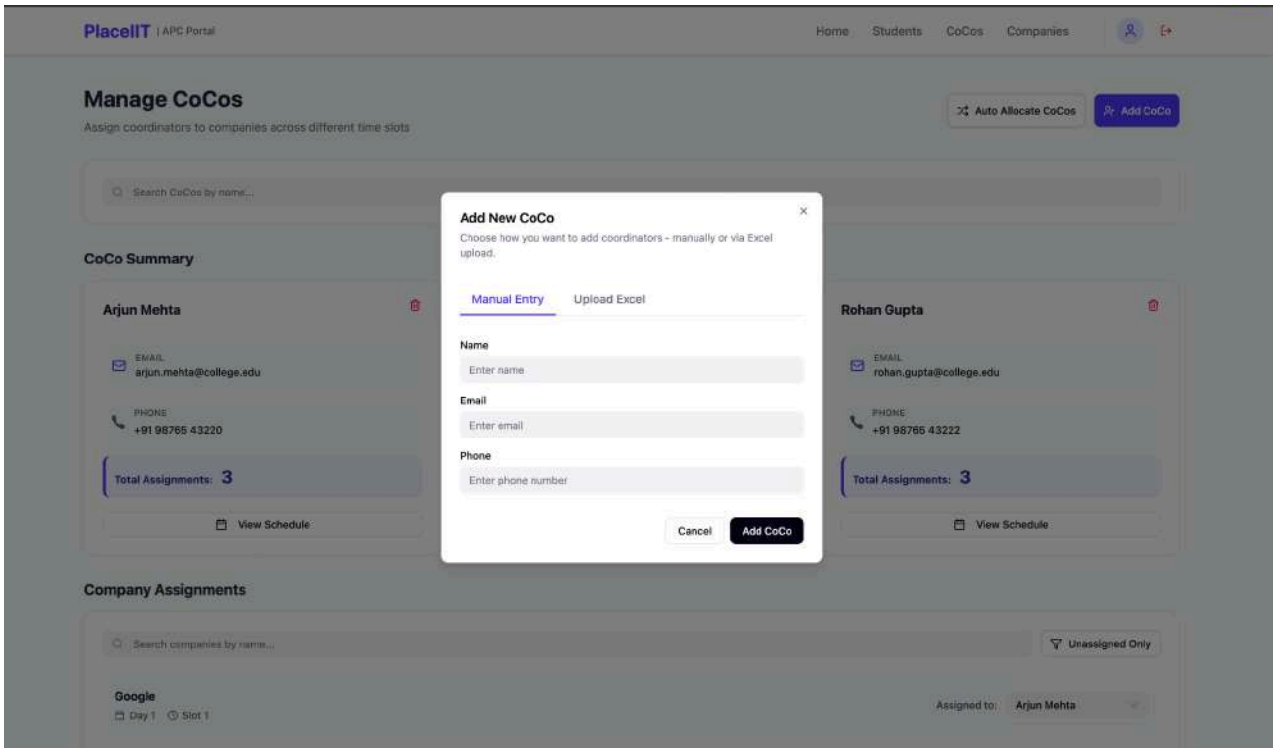
1.2.4.3 Student Detail



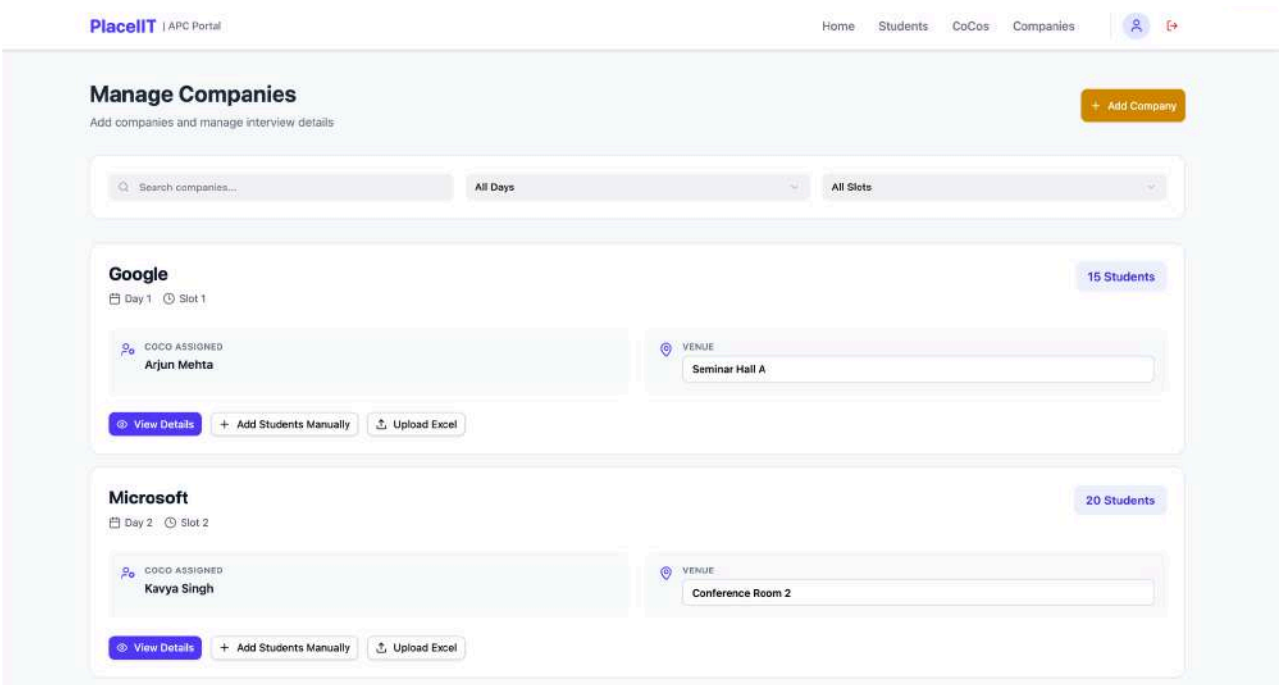
1.2.4.4 Manage CoCo



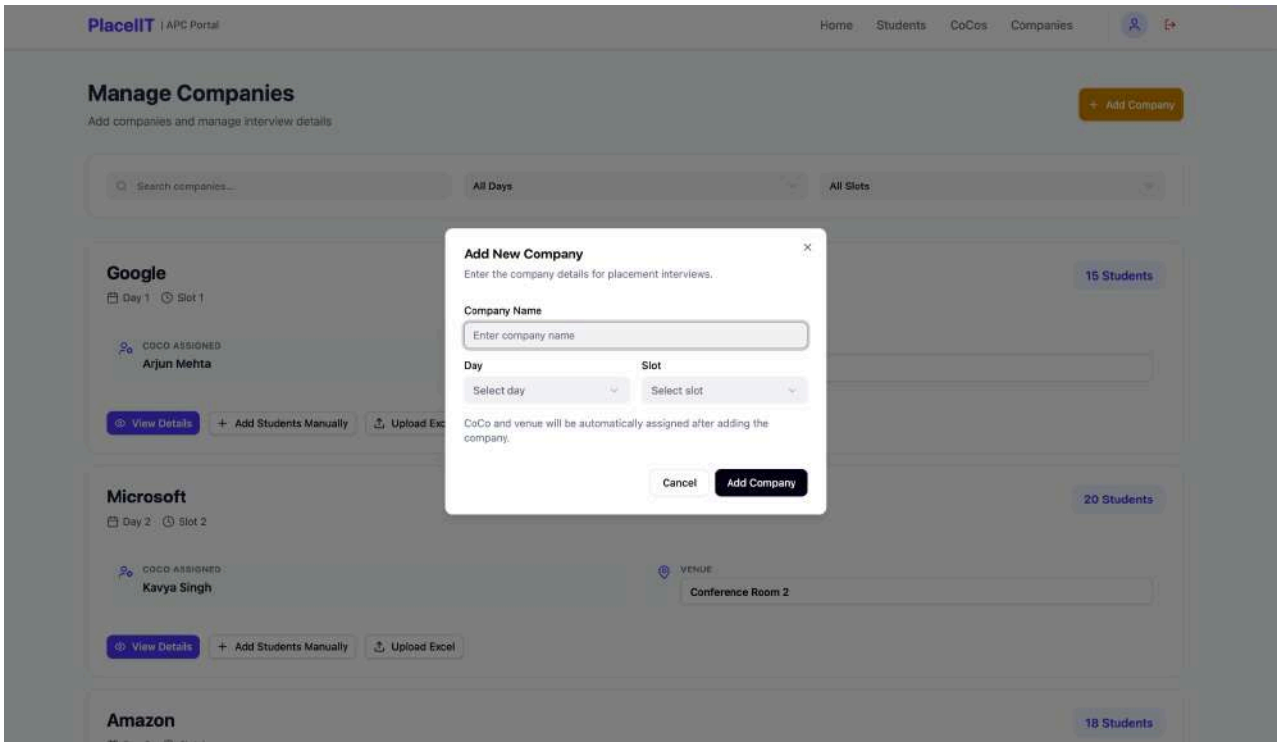
1.2.4.5 Add CoCo



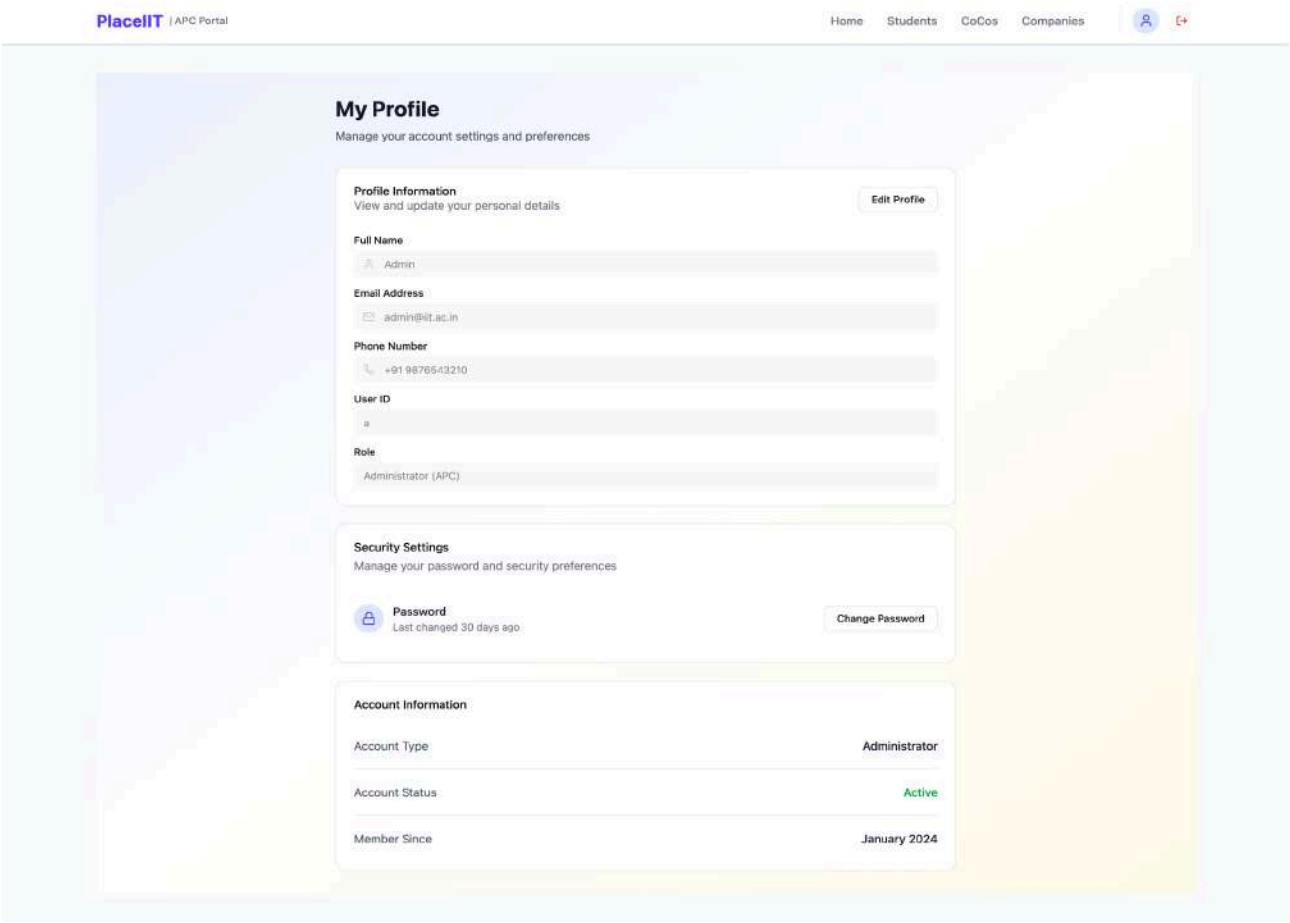
1.2.4.6 Manage Company



1.2.4.7 Add Company



1.2.4.8 View Profile



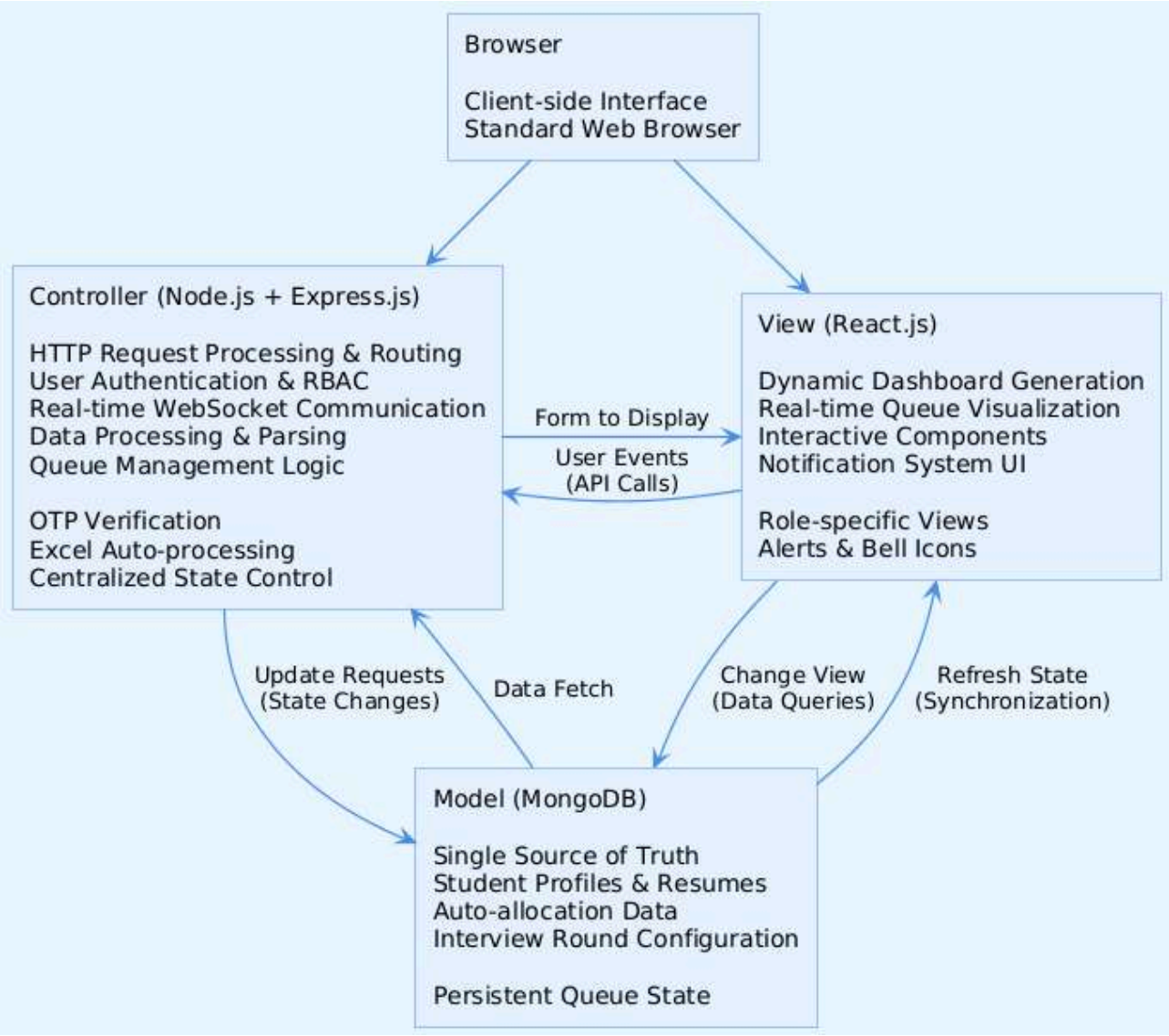
● Architecture Design

2.1 Architectural Overview

PlaceIIT is a web-based real-time placement coordination system designed to be accessed through standard web browsers by students, company coordinators, and administrators.
The system follows the **Model–View–Controller (MVC)** architectural pattern to ensure a clear separation between user interface, business logic, and data management.

This architecture supports:

- Real-time interview queue synchronization
- Role-based access control
- Concurrent access by multiple users



2.2 Model–View–Controller Components

2.2.1 Model

The Model layer manages all persistent application data and acts as the single source of truth for the system..

Responsibilities:

- Store student profiles, resumes, and interview configurations
- Maintain persistent interview queue states
- Manage company data and interview rounds

Implementation:

- Implemented using MongoDB
- Supports flexible schemas required for dynamic interview workflows
- Ensures consistency during concurrent read/write operations

2.2.2 View

The View layer is responsible for presenting data to users and capturing user interactions.

Responsibilities:

- Render role-specific dashboards (Student, CoCo, Admin)
- Display real-time queue positions and interview status
- Provide interactive UI components such as join queue, walk-in, and notifications

Implementation:

- Built using React.js
- Automatically updates UI based on state changes received from the Controller
- Supports real-time visualization without page reloads

2.2.3 Controller

The Controller acts as the central coordination layer between the View and the Model.

Responsibilities:

- Handle HTTP requests and API routing

- Enforce role-based access control
- Process business logic for queue management and interviews
- Handle Excel-based bulk uploads and data processing
- Manage real-time communication using WebSockets

Implementation:

- Built using Node.js with Express.js
- Uses WebSockets for real-time state propagation
- Ensures controlled and validated updates to the Model

2.3 System Interaction Flow

1. Users interact with the system through a web browser
2. The View sends user actions to the Controller as API calls
3. The Controller processes requests and updates the Model
4. Updated state is propagated back to all relevant Views
5. The View refreshes the UI to reflect real-time changes

This flow guarantees consistent, synchronized, and low-latency updates across all users.

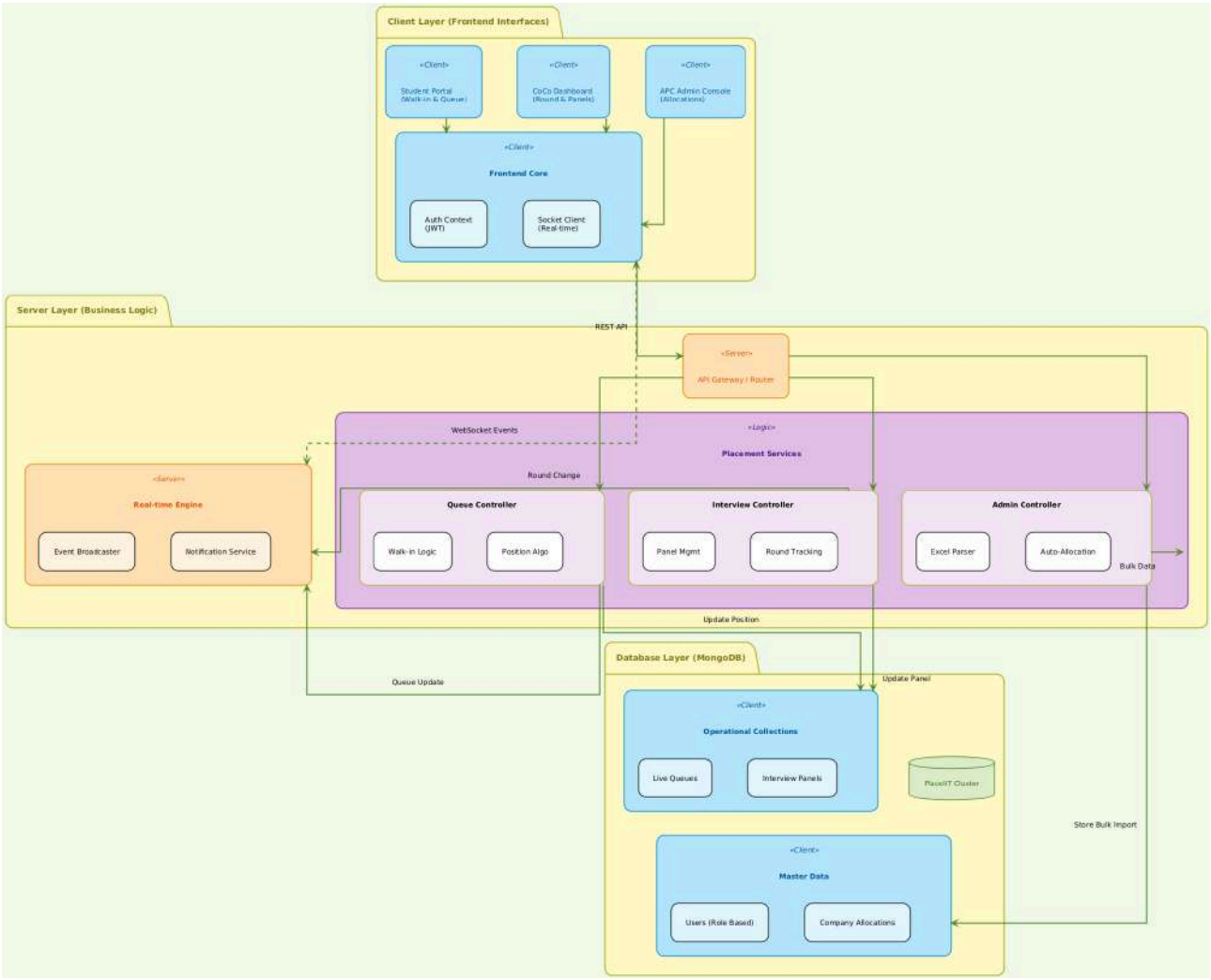
2.4 Architectural Rationale

The MVC architecture was chosen because it:

- Separates concerns clearly
- Simplifies real-time state management
- Improves scalability and maintainability
- Supports multiple user roles efficiently

This design ensures that PlaceIIT remains reliable and responsive during high-load placement scenarios.

2.5 Tech-Stack Architecture



The technical architecture diagram illustrates the high-level structure of the PlaceIIT Campus Placement System. It outlines the primary components across the client-side interface, server-side business logic, database layer, and the crucial real-time communication infrastructure. This diagram demonstrates how data flows securely between these distinct layers to fulfill distinct user requests and maintain synchronous operational states.

Frontend: Built using a Component-Based JavaScript Framework to provide a dynamic and responsive user experience. The frontend architecture is segmented into distinct role-based modules—Student Portal, CoCo Dashboard, and APC Console—ensuring secure, role-specific feature access. Global application state (authentication, live queue status, UI alerts) is managed centrally using a State Management Library. The network layer utilizes an Asynchronous HTTP Client for standard REST API interactions and an integrated WebSocket Client to handle real-time event listeners.

Backend: Powered by an Event-Driven Runtime Environment to handle robust RESTful API services. Requests first pass through a Middleware Pipeline that handles security (Token-based Authentication, Role-Based Access Control) and specific tasks like Multipart File Processing for parsing large datasets. The core functionality is divided into specialized service modules: a Queue Engine (for FCFS algorithms and walk-ins), an Interview Engine (for panel management and tracking), and an Admin Engine (for auto-allocation logic).

Real-time Gateway: A dedicated WebSocket Protocol Server runs alongside the main application to manage bi-directional connections. This is the backbone of PlaceIIT's live features, responsible for emitting events for instant notifications, real-time queue position updates, and dynamic dashboard refreshes without requiring page reloads.

Database: Utilizing a NoSQL Document Store as the persistent data layer due to its flexibility with complex data structures. The data models are organized into schemas for User Data (profiles, roles), Operational Data (transient data like live queue arrays and active panel sessions), and Master Data (static data like company details and interview rounds).

- **This architecture should meet the following critical non-functional requirements:**

Scalability: The architecture is engineered to be highly scalable to accommodate the surge of traffic during placement season. By utilizing the MERN stack (MongoDB, Express.js, React.js, and Node.js), the system can efficiently handle a rapidly increasing number of student and recruiter concurrent sessions. The backend (Node.js with Express.js) and the database (MongoDB) are designed to be scaled horizontally by deploying multiple instances behind a load balancer. Furthermore, MongoDB's sharding mechanism allows for distributing large datasets across multiple servers, ensuring consistent performance at scale.

Performance: The non-blocking, event-driven architecture of the MERN stack ensures high performance and responsiveness, which is crucial for real-time queue management. The use of efficient RESTful APIs (alongside WebSockets) for communication between the frontend (React.js) and backend (Express.js) minimizes network latency.

Security & Privacy: Security is a top priority, as the PlaceIIT platform handles sensitive academic records and recruitment data. The architecture implements rigorous security measures, including the protection of sensitive personal information and the enforcement of strict Role-Based Access Control (RBAC). This is reinforced by a robust authentication system that strictly verifies the IITK email ID of users, preventing unauthorized access. All security protocols are implemented centrally via the backend framework to ensure uniform enforcement.

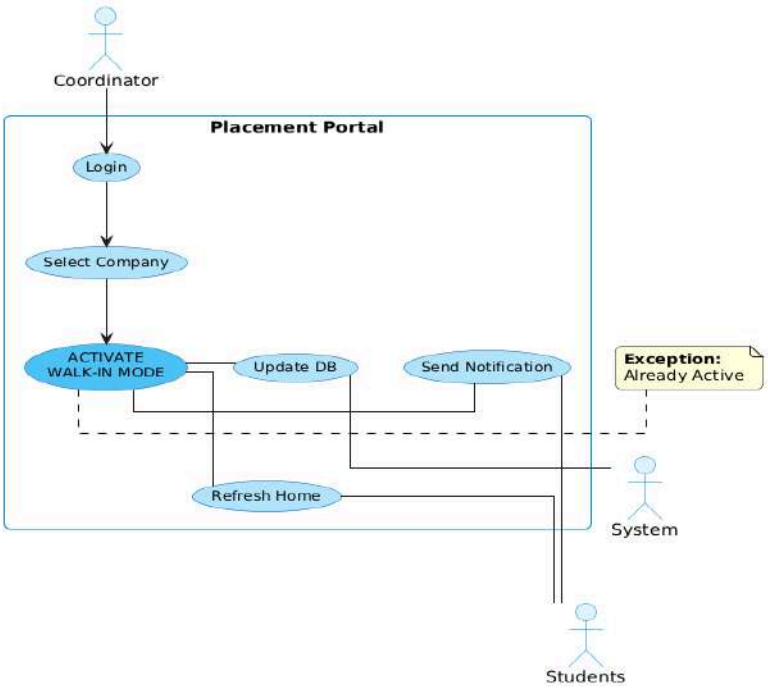
Interoperability: The chosen technology stack ensures seamless compatibility across diverse platforms and devices. React.js provides a responsive User Interface (UI) that functions consistently across both web and mobile browsers, allowing students to track updates on the go. The backend's standardized API structure allows for easy integration with future third-party services (e.g., external assessment platforms) if required. The uniform use of JavaScript throughout the entire stack (frontend, backend, and database interactions) streamlines the development process and ensures cohesive code maintenance.

Reliability: To ensure the system remains operational during critical recruitment drives, the architecture prioritizes robust design and fault tolerance. Through rigorous unit testing and beta testing phases, we aim to deliver a stable experience. We plan to implement a Continuous Integration and Deployment (CI/CD) pipeline within the architecture to automatically detect bugs and prevent regressions. Additionally, comprehensive and up-to-date documentation will be maintained to ensure the system is easy to use, maintain, and dynamically update without disrupting service availability.

● Object-Oriented Design

3.1 Use Case Diagrams

3.1.1 Use Case #1 - Activate Walk-In Feature



Purpose: Enables walk-in mode for a recruitment drive and instantly notifies eligible students.

Actors: Coordinator (CoCo), System, Students.

Preconditions: CoCo is logged in; recruitment drive is scheduled for the current day.

Process Flow:

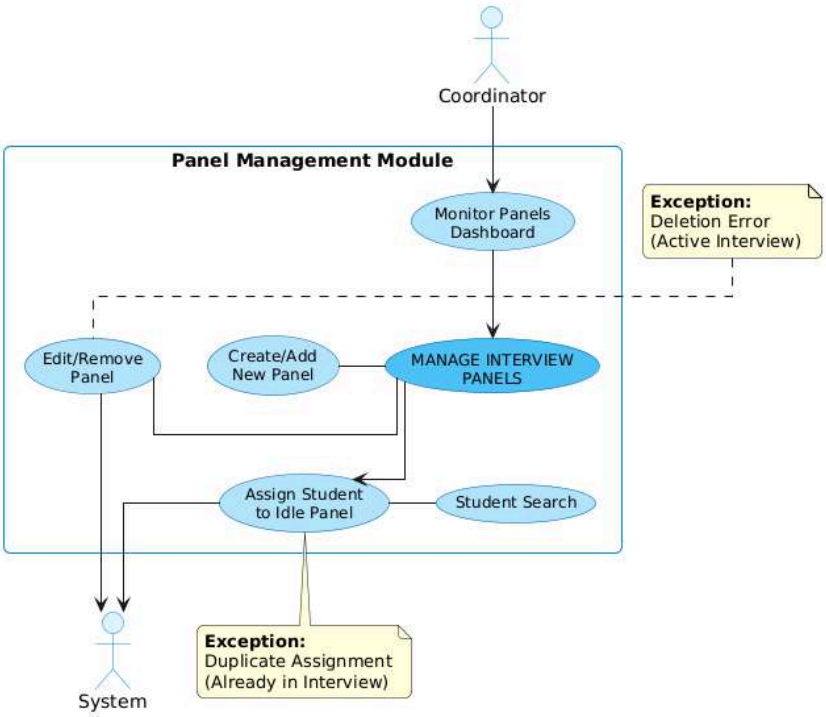
- **Trigger:** CoCo selects the company and clicks "Activate Walk-In Mode."
- **System Actions:** The system simultaneously updates the database status to "Active," broadcasts push notifications to students, and refreshes the CoCo dashboard.

Post Conditions: The company becomes visible in the Student "Walk-In" section, and its notifications are delivered to the eligible students.

Exceptions:

- **Already Active:** The system prevents duplicate activations.
- **Network Timeout:** Error handling for notification failures.

3.1.2 Use Case #2 - Manage Interview Panels



Purpose: To manage interview panels (monitor, create, edit) and assign students to idle panels for efficient resource utilization.

Actors: Coordinator (CoCo), System.

Preconditions: CoCo is on the Panel Dashboard; interview rounds are defined; eligible students are waiting.

Process Flow:

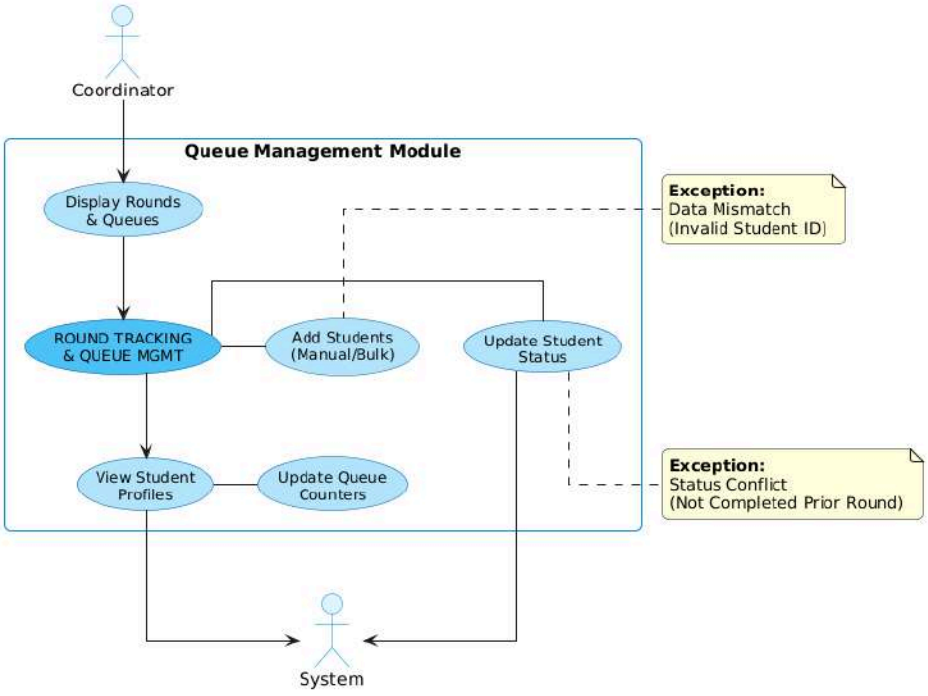
- **Monitor:** CoCo tracks real-time panel status (members, location) via the dashboard.
- **Manage:** CoCo creates new panels or modifies existing ones as needed.
- **Assign:** CoCo identifies idle panels and assigns waiting students.
- **System Actions:** System updates availability and marks the student as "In Interview."

Post Conditions: Panel configuration updates in DB; student status changes to "In Interview."

Exceptions:

- **Duplicate Assignment:** Blocks assigning students already in an interview.
- **Deletion Error:** Prevents deleting panels with active interviews.

3.1.3 Use Case #3 - Round Tracking & Queue Management



Purpose: To manage candidate flow through sequential interview rounds and track real-time status (In Queue, Interviewing, Completed).

Actors: Coordinator (CoCo), System.

Preconditions: CoCo is logged in; a shortlist of students exists for the specific day/slot.

Process Flow:

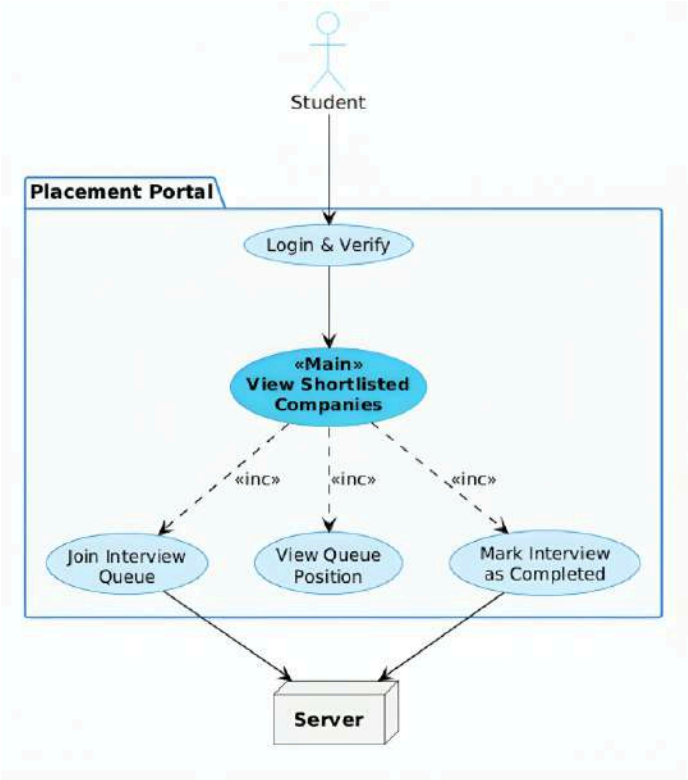
- **Monitor:** CoCo views the column-based layout displaying live queues and round summary counters.
- **Manage:** CoCo adds students (manually or via bulk selection) to the recruitment pipeline.
- **Update:** CoCo moves students between stages or updates their status.
- **System Actions:** System validates the move, updates status badges, and refreshes the queue counters.

Post Conditions: Student status updates (e.g., "In Queue" → "Completed"); round counters reflect current numbers.

Exceptions:

- **Data Mismatch:** Invalid Student ID entered during manual addition.
- **Status Conflict:** Preventing a student from moving to "Round 2" without completing "Round 1."

3.1.4 Use Case #4 - Viewing Shortlisted Companies



Purpose: View shortlisted companies, manage queues, and track interview status.

Actors: Student, Server.

Preconditions: Student is logged in; shortlist data is available.

Process Flow:

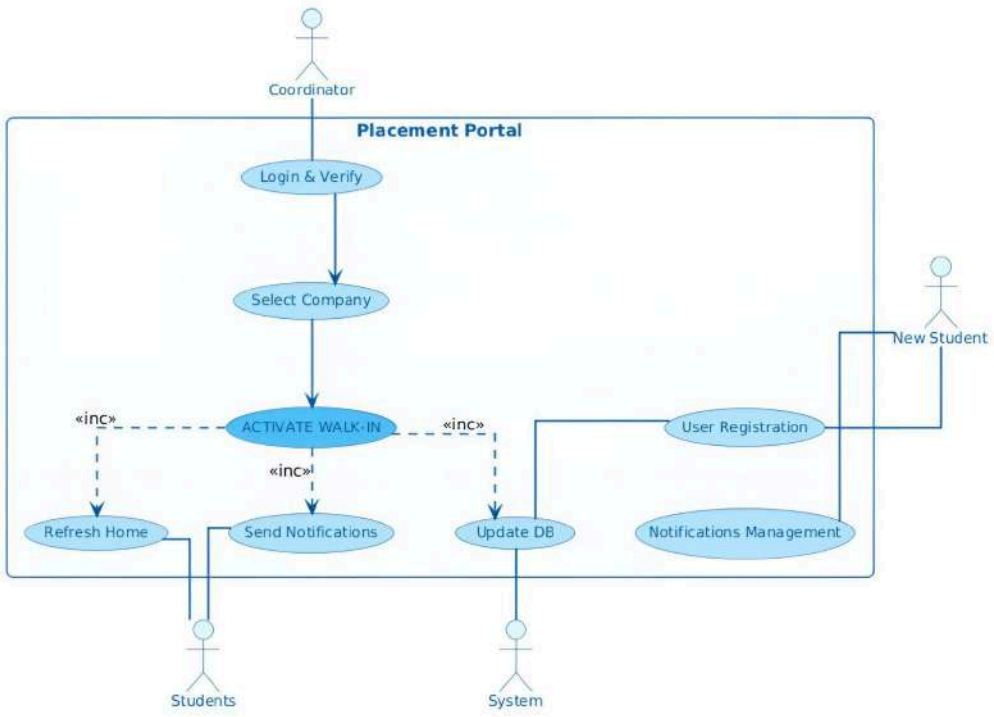
- **View & Join:** Student views the company list and clicks "Join Queue" for active sessions.
- **Track & Complete:** Student monitors live position and marks the interview "Completed" when done.

Post Conditions: Database updates status to "In Queue" or "Completed."

Exceptions:

- **No Shortlist:** Alerts if no companies are found.
- **Queue Inactive:** Blocks entry if closed or full.

3.1.5 Use Case #5 - User Registration



Purpose: To enable new students to create an account on the PlaceIIT Campus Placement System.

Actors: New Student, System.

Preconditions: None.

Process Flow:

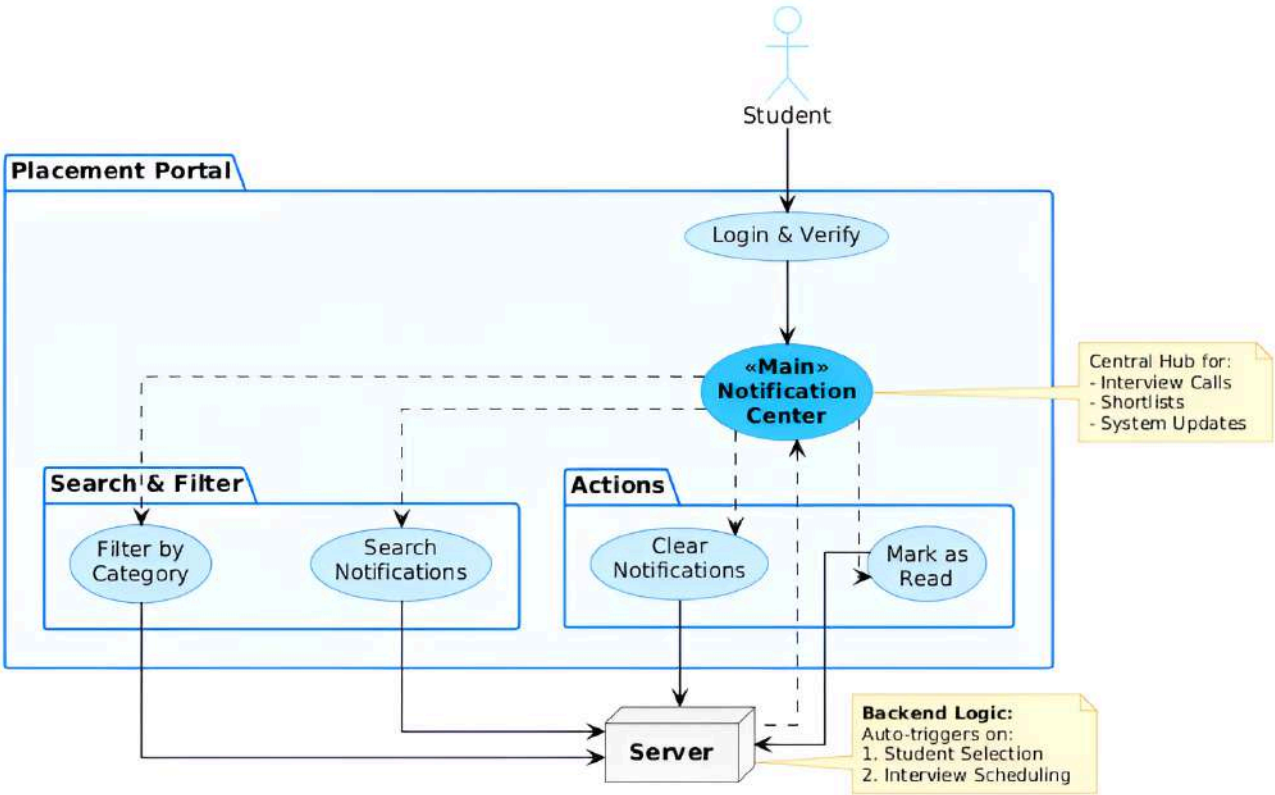
- **Access:** Student navigates to the login page and selects the "Student" role.
- **Input:** Student enters their Name, IITK Email ID, and Password.
- **Validation:** System verifies the email is a valid IITK domain and checks for existing accounts.
- **Creation:** The system creates the user profile and updates the database.

Post Conditions: Student account is successfully created and stored in the system.

Exceptions:

- **Invalid Domain:** System rejects emails not ending in the IITK domain.
- **Duplicate User:** System prevents registration if the email is already in use.

3.1.6 Use Case #6 - Notifications Management



Purpose: Notifies students of real-time updates like interview calls, queue alerts, and venue changes.

Actors: Student.

Preconditions: Student is logged in; placement events exist.

Process Flow:

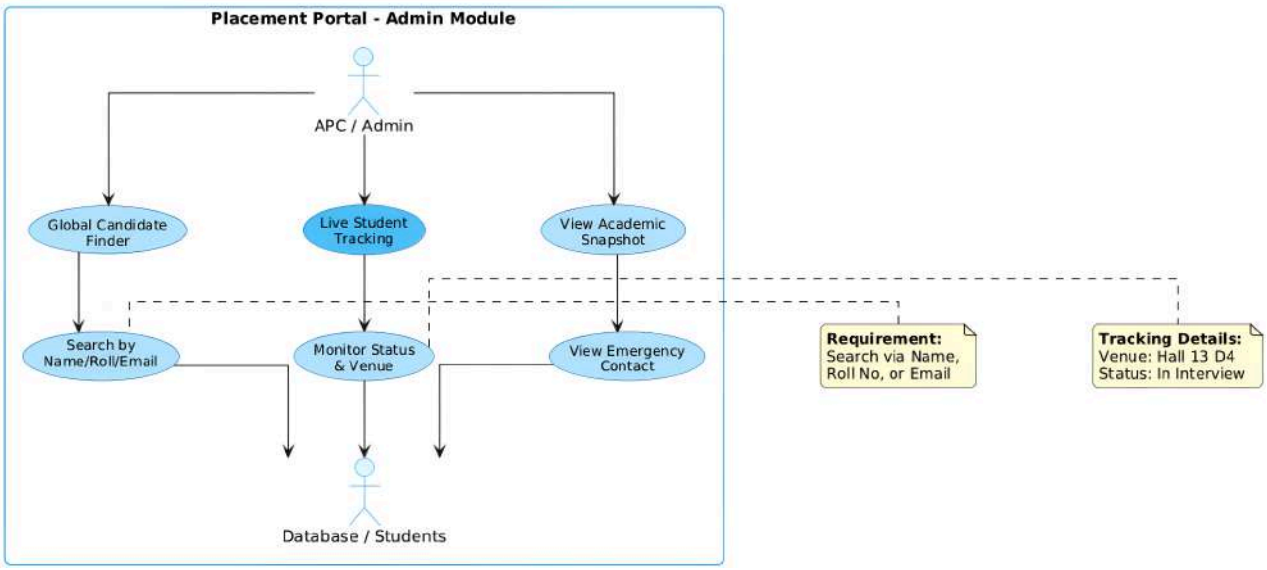
- **Access:** Student opens the "Notification Center" hub.
- **Filter & Search:** Student filters by category or searches keywords.
- **Manage:** Student marks items as read or clears the feed.
- **System Actions:** Backend triggers alerts for selection or scheduling events.

Post Conditions: Notifications are viewed or cleared; unread counts update.

Exceptions:

- **No Data:** Shows "No notifications found" if empty.
- **Search Failure:** Shows "No matching notifications" for failed searches.

3.1.7 Use Case #7 - Search and Track Student



Purpose: To locate candidates and track their live interview status, venue location, and contact details.

Actors: APC Admin.

Preconditions: Admin is logged in; student database is populated.

Process Flow:

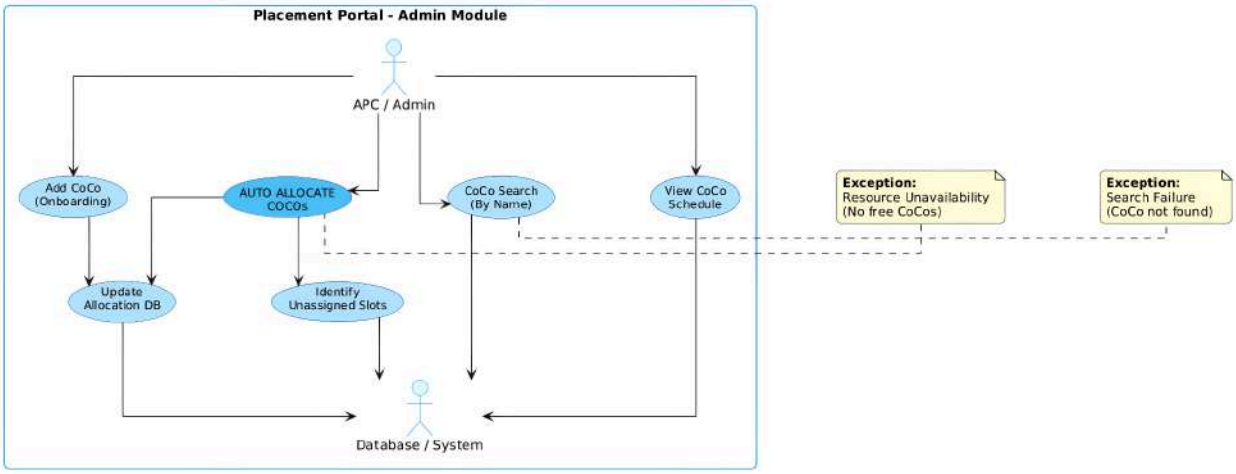
- **Search:** Admin uses the "Global Candidate Finder" to search by Name, Roll No, or Email.
- **Track:** System displays real-time status (e.g., "In Interview") and specific venue (e.g., "Hall 13").
- **View Details:** Admin accesses academic snapshots and emergency contact information.

Post Conditions: Student profile is displayed with current status, location, and resume access.

Exceptions:

- **Not Found:** Displays an error for an invalid Roll Number or Name.
- **Data Unavailable:** Live status or location fails to sync.
- **File Error:** Resume file is missing or corrupted.

3.1.8 Use Case #8 - Manage Company Coordinators (CoCos)



Purpose: To streamline the onboarding of Coordinators (CoCos) and efficiently manage their assignments through manual selection or automated allocation algorithms.

Actors: APC / Admin.

Preconditions: The Admin is successfully logged in, and a valid list of companies exists in the database for allocation.

Process Flow:

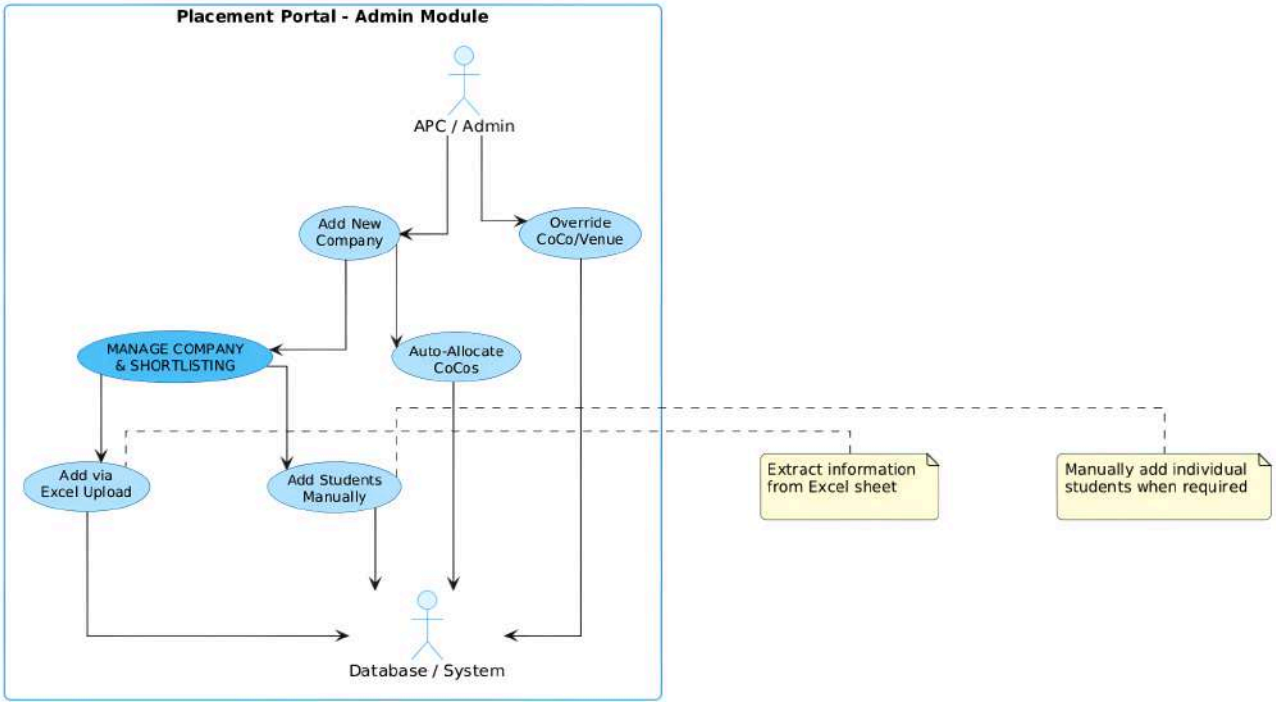
- **Manage:** The Admin can onboard new CoCos or search the existing database by name to find specific personnel.
- **Allocate:** The system's auto-allocation feature fills empty slots by assigning available CoCos to companies that lack coverage.
- **View & Monitor:** The Admin can view detailed schedules for individual CoCos to ensure balanced workloads.
- **System Actions:** The backend updates the allocation database in real-time and retrieves specific slot details for display.

Post Conditions: CoCos are successfully assigned to companies; updated schedules are viewable.

Exceptions:

- **Resource Unavailability:** The system flags an error if no free CoCos are available for auto-allocation.
- **Search Failure:** An error message is displayed if the searched CoCo name is not found.

3.1.9 Use Case #9 - Manage Company and Shortlisting



Purpose: Adds companies with schedules, manages shortlists, and overrides assignments.

Actors: APC / Admin.

Preconditions: Admin is logged in; valid Excel format for uploads.

Process Flow:

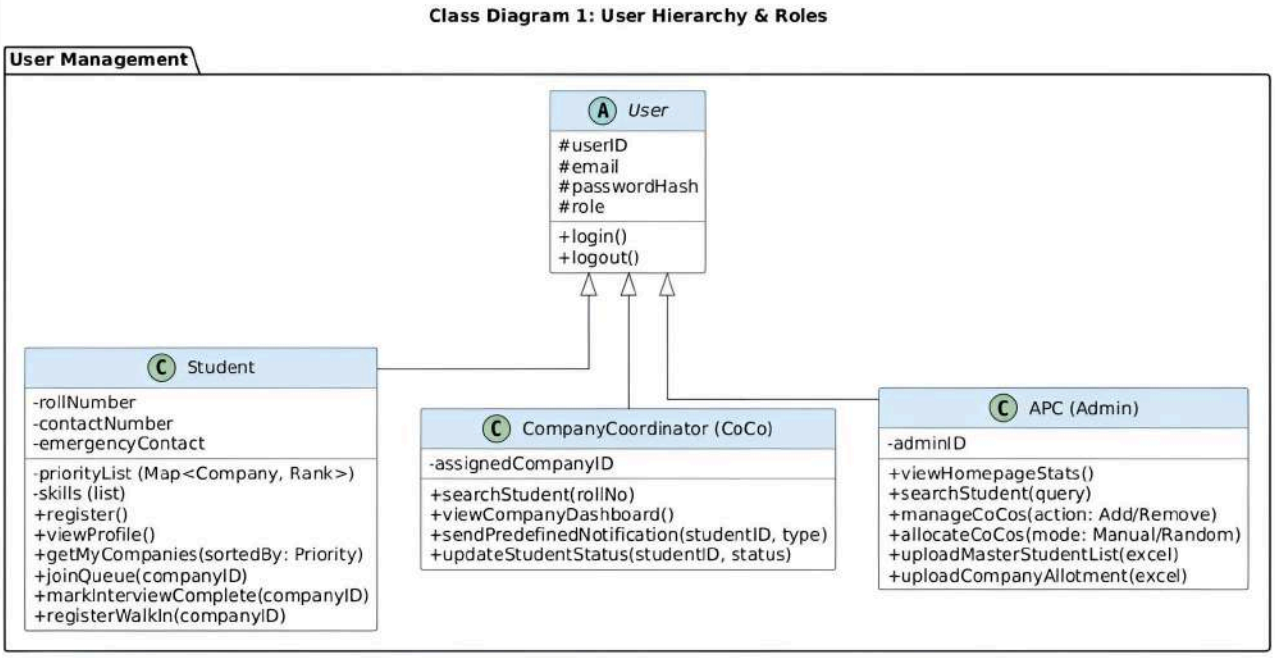
- **Add Company:** Admin creates a company entry with Day/Slot configuration.
- **Shortlist:** Admin populates student lists via Excel upload or manual entry.
- **Auto-Allocate:** System triggers automatic CoCo allocation.
- **Override:** Admin manually updates Venue or CoCo assignments if needed.

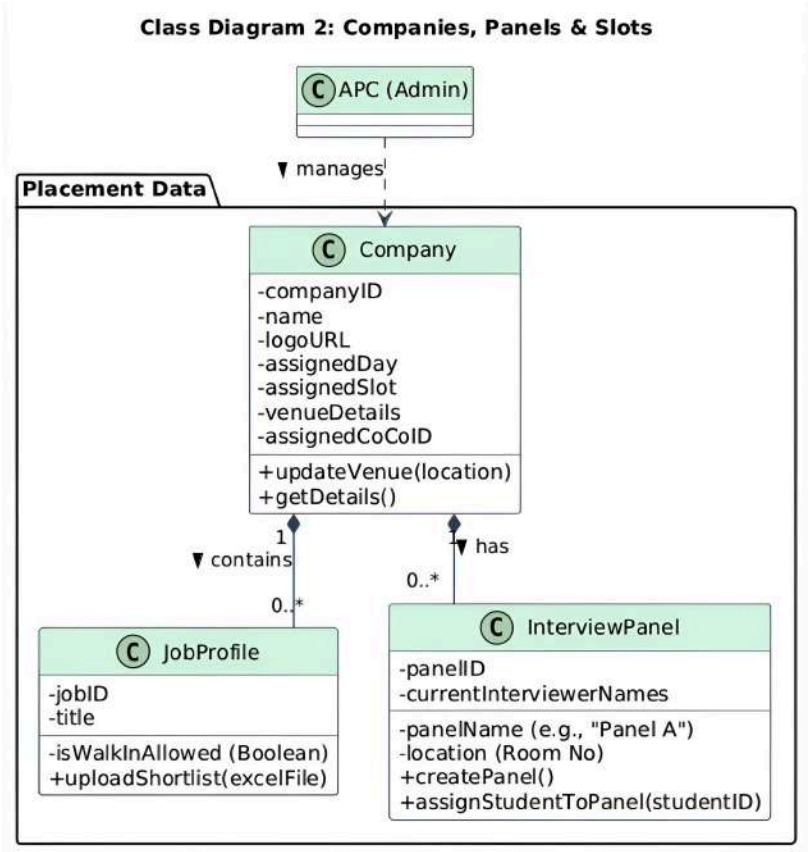
Post Conditions: Company created; shortlist saved; assignments updated.

Exceptions:

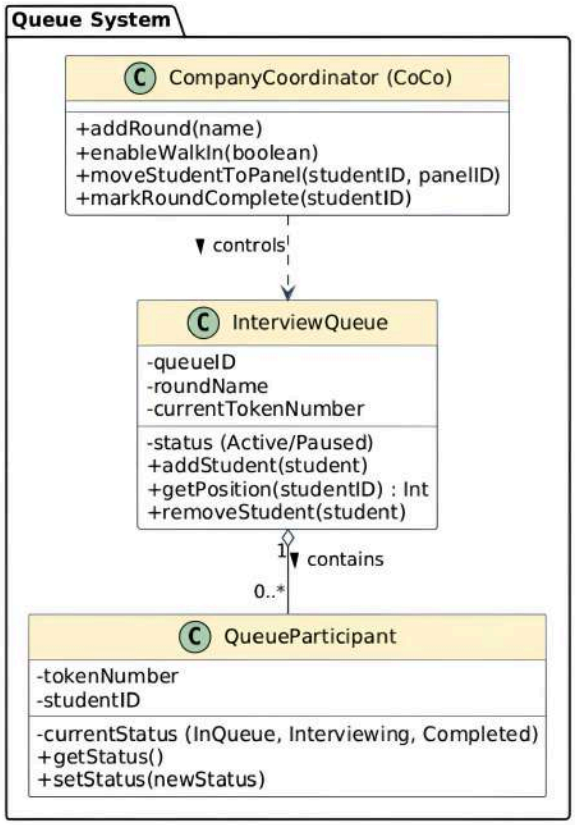
- **Format Error:** Fails if the Excel schema is invalid.
- **Scheduling Conflict:** Fails if manual venue change overlaps with existing bookings.

3.2 Class Diagrams

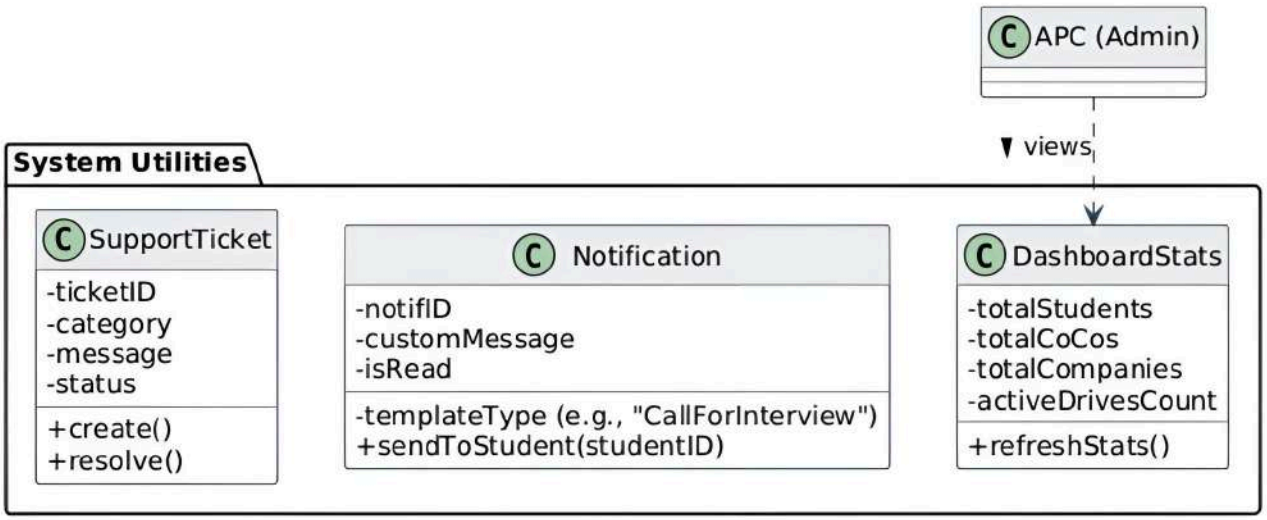




Class Diagram 3: Interview Queue & Status



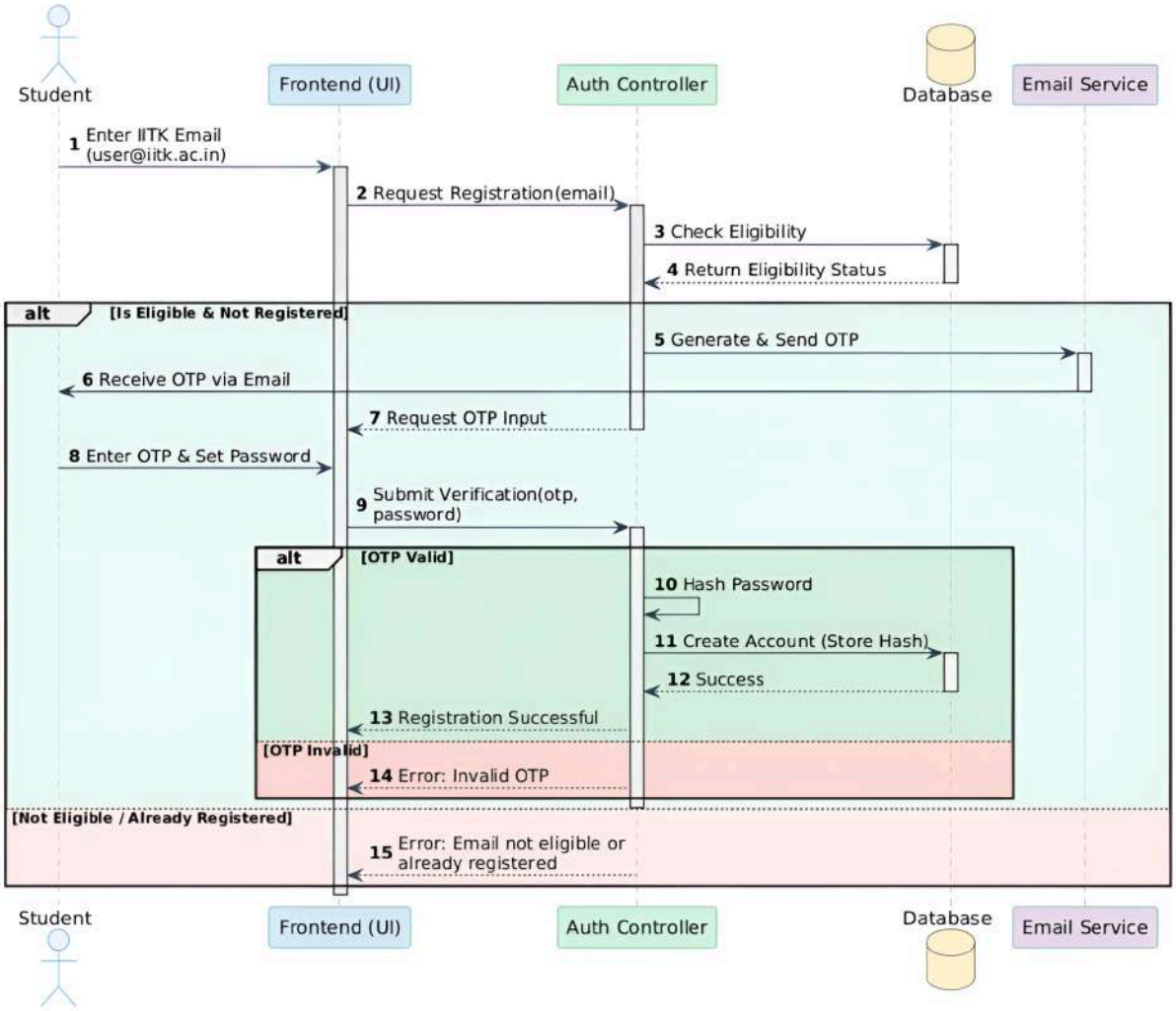
Class Diagram 4: Dashboard & Utilities



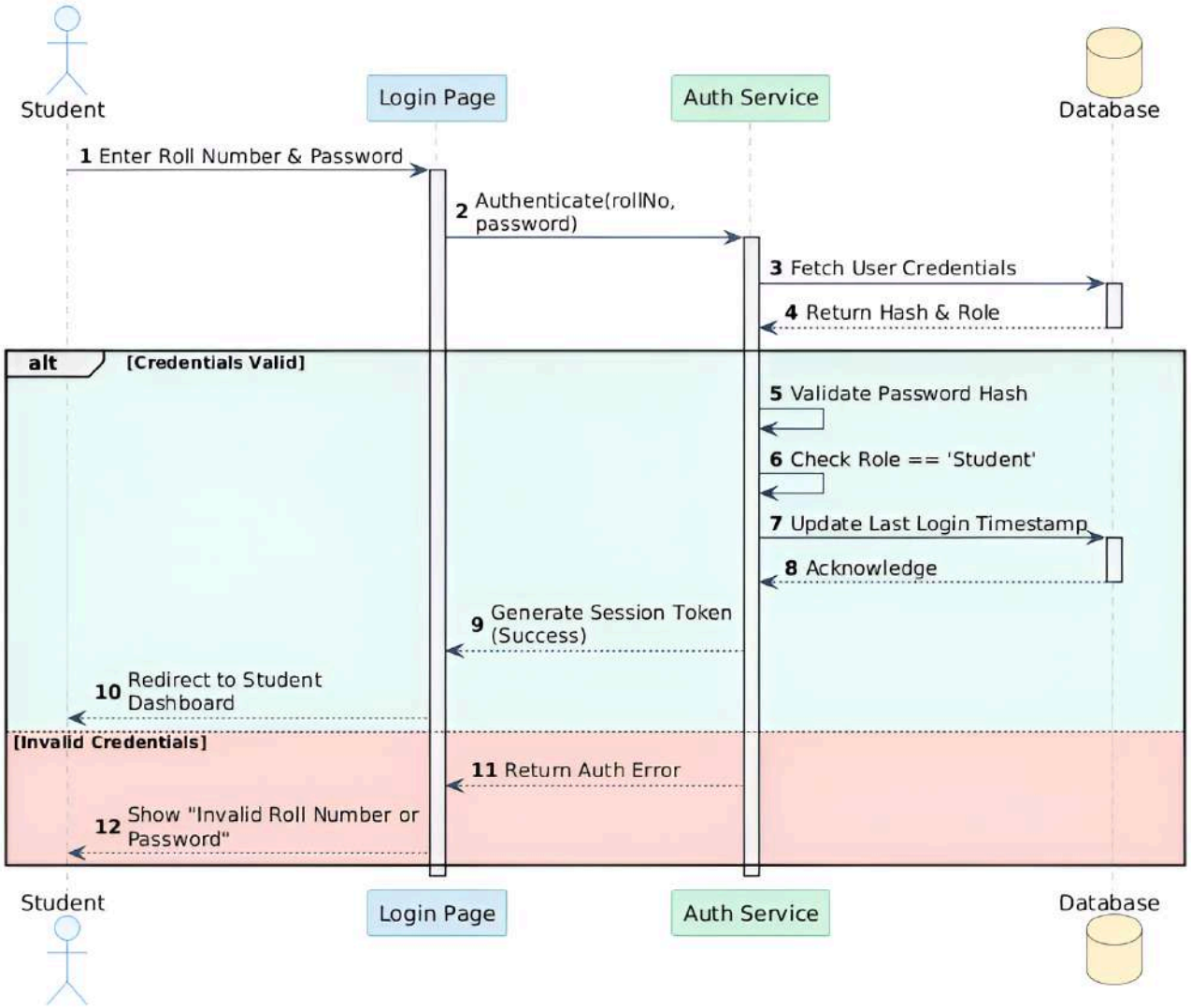
3.3 Sequence Diagrams

I. User : Student

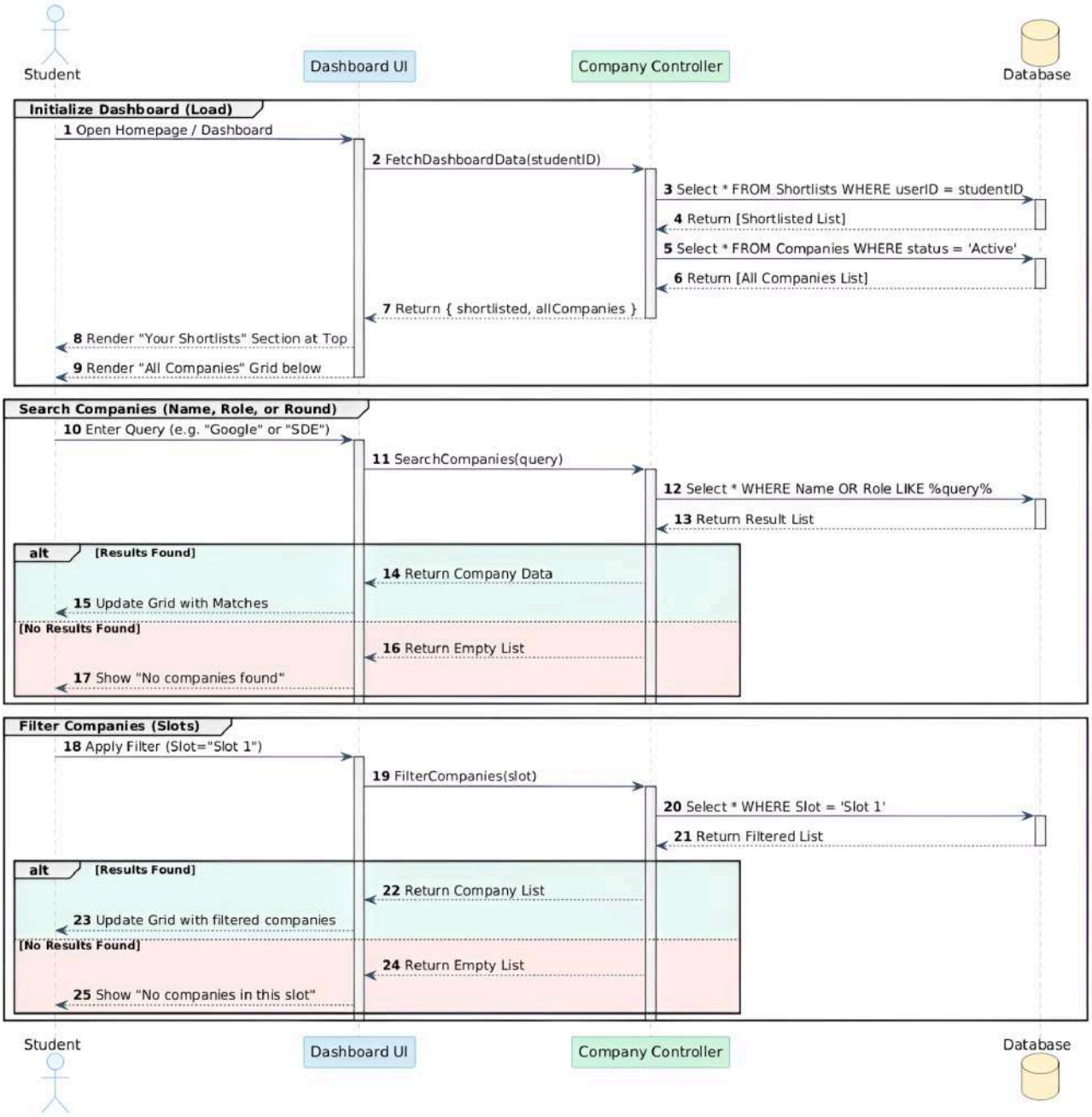
A. Student Registration and Verification



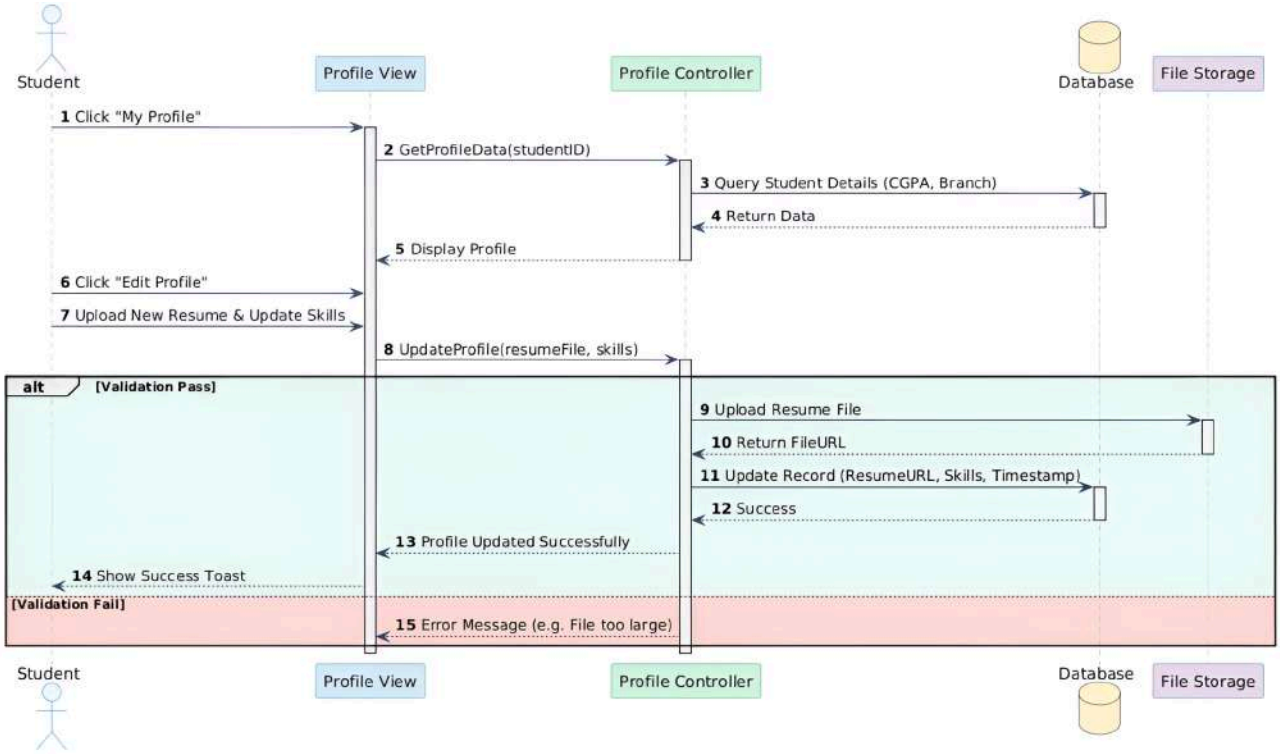
B. Student Login Process



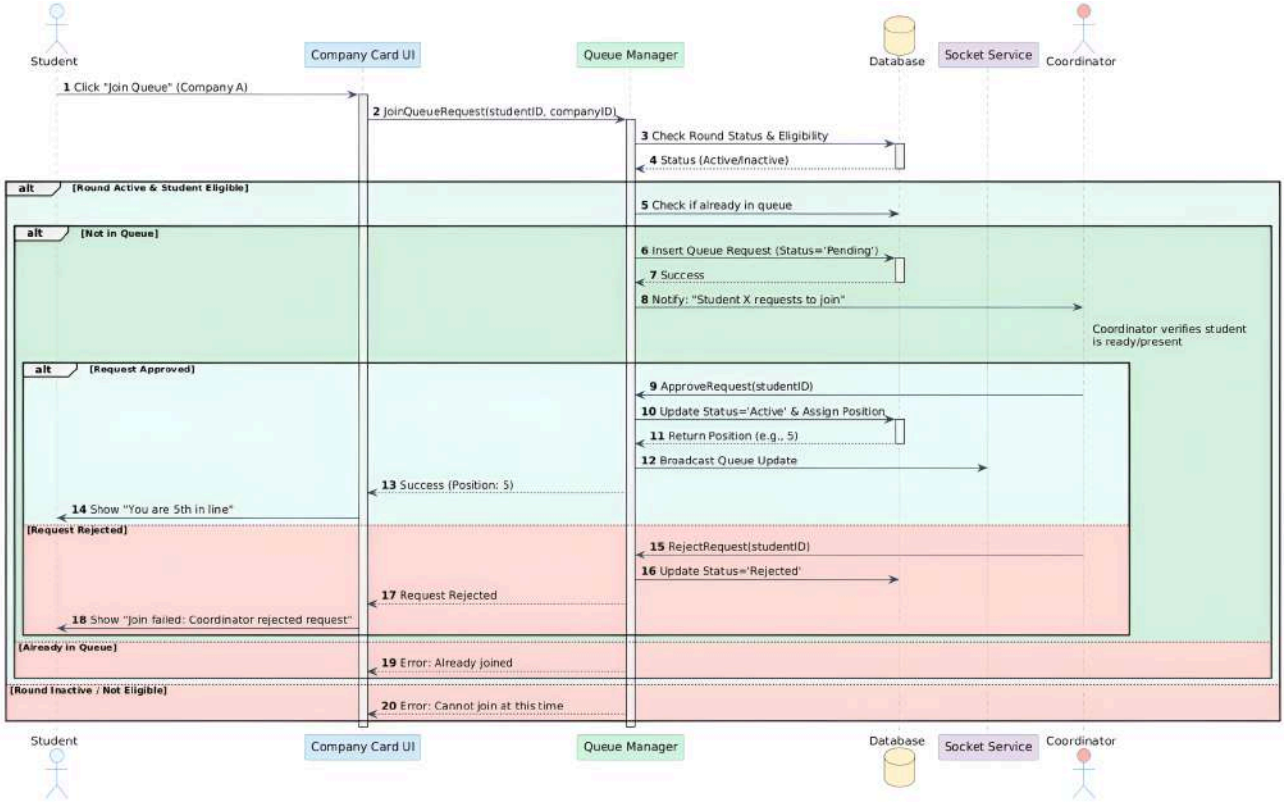
C. Student Homepage



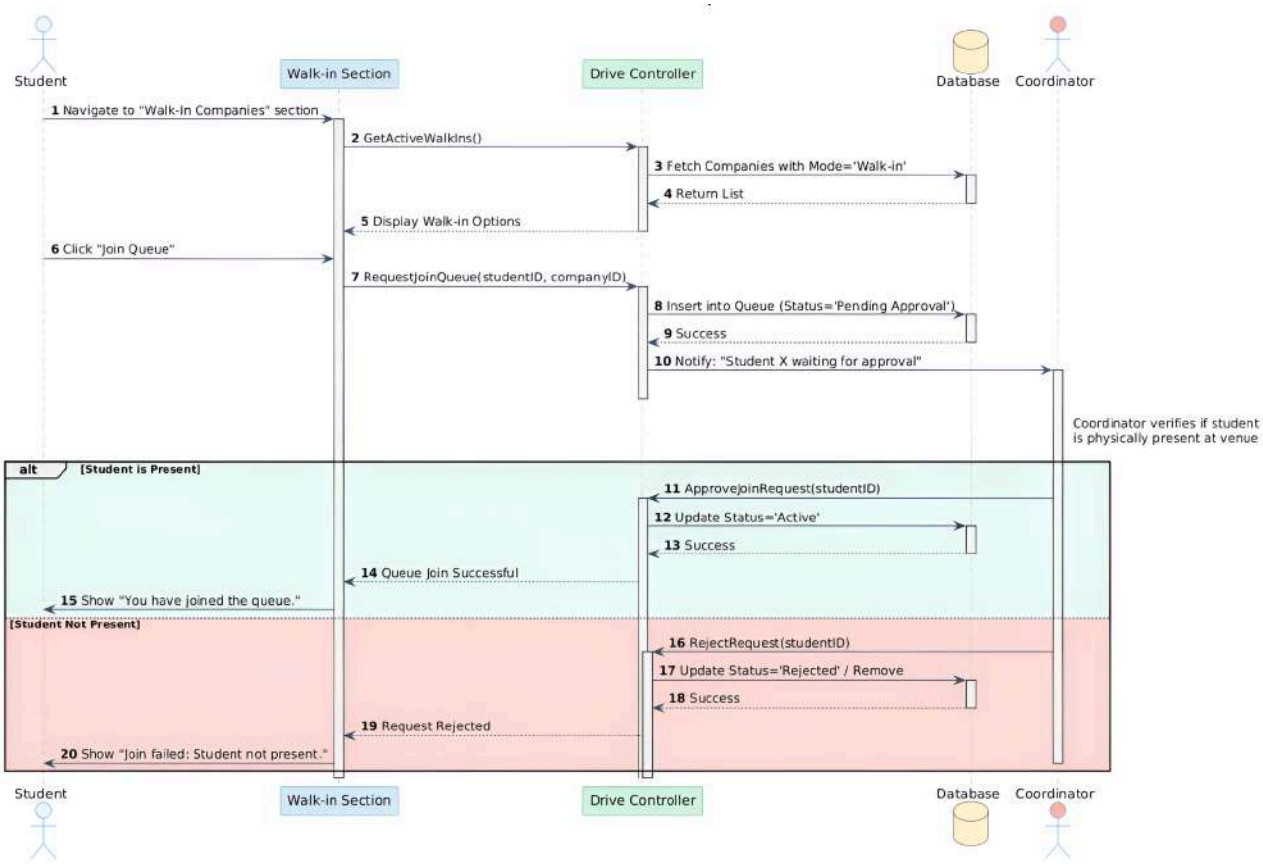
D. View and Edit Profile



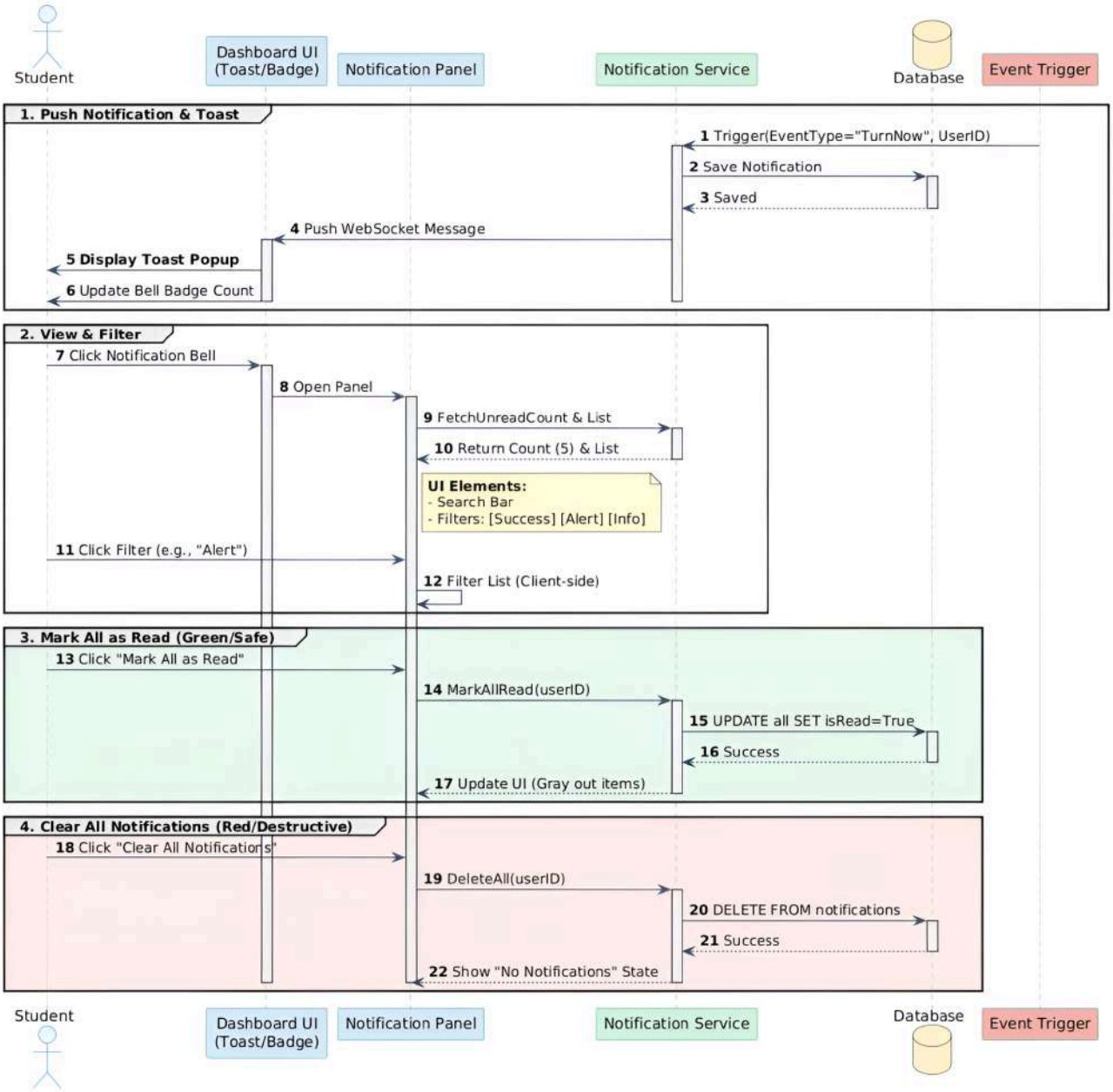
E. Join Interview Queue



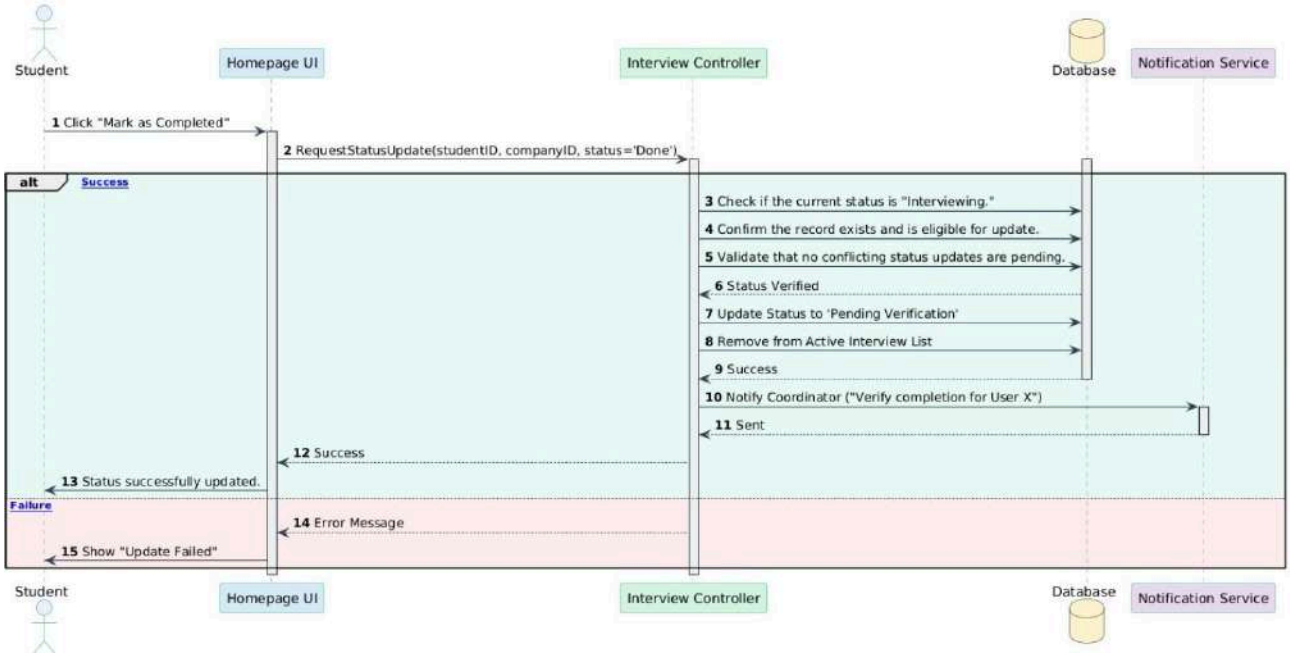
F. Walk - In Interview Section



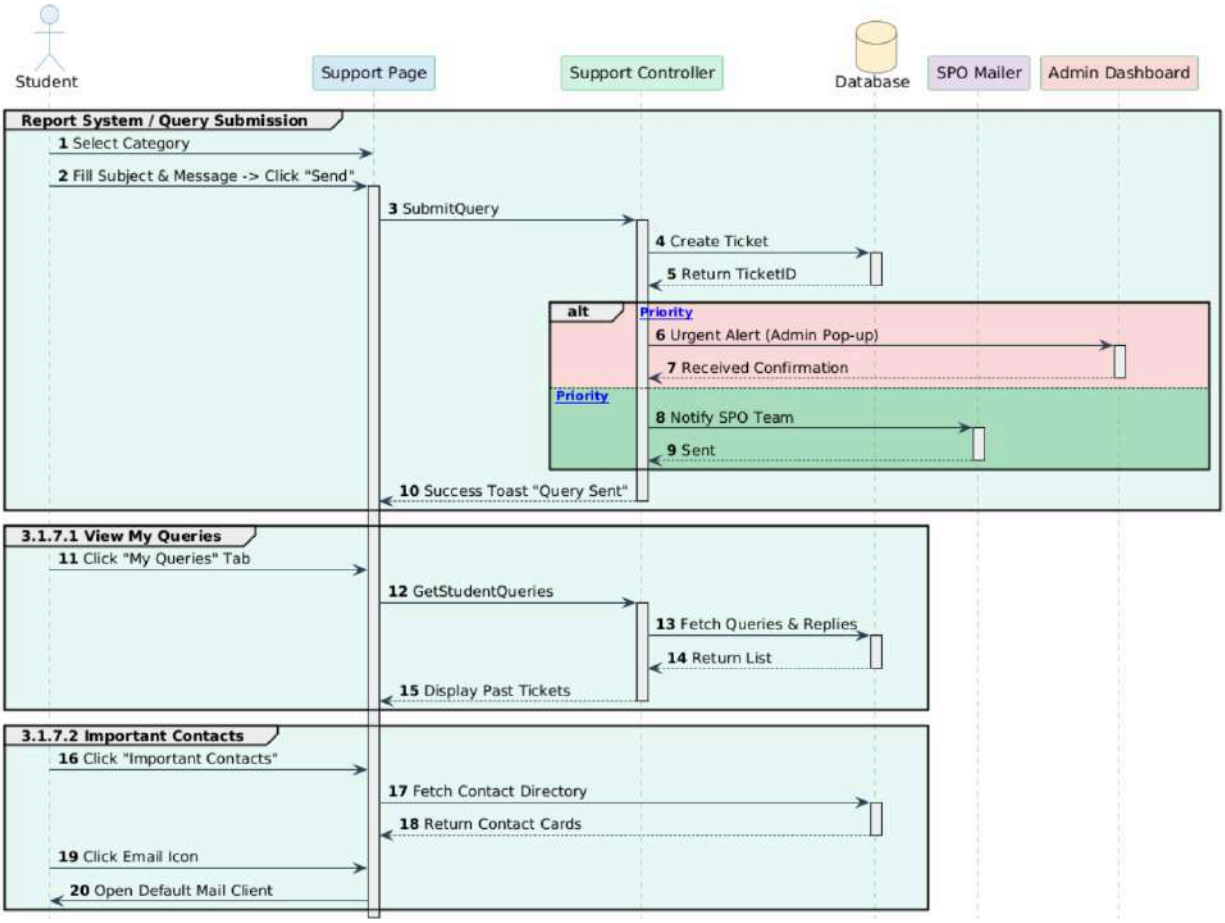
G. Notification Management



H. Mark interview completed

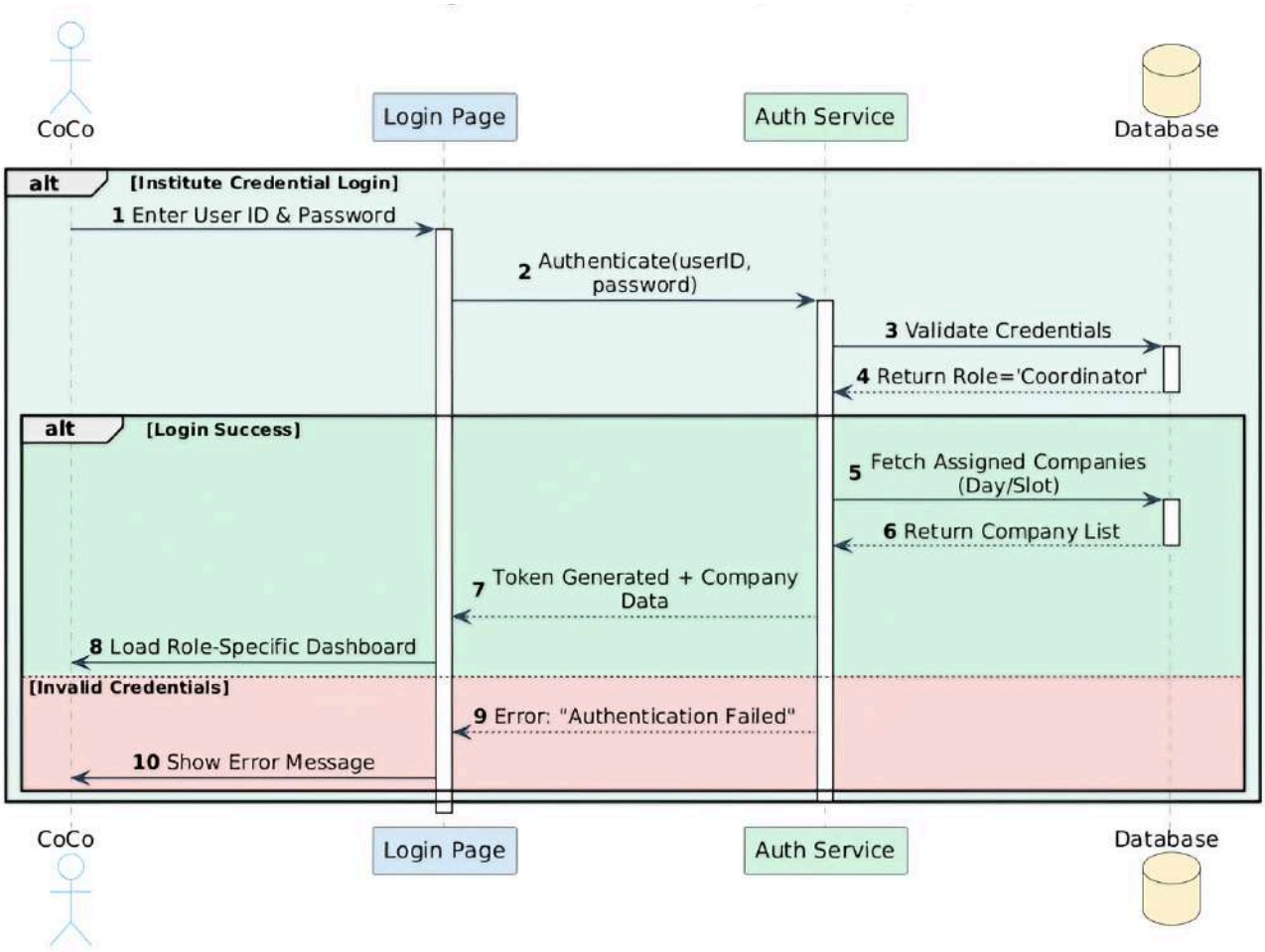


I. Contact us

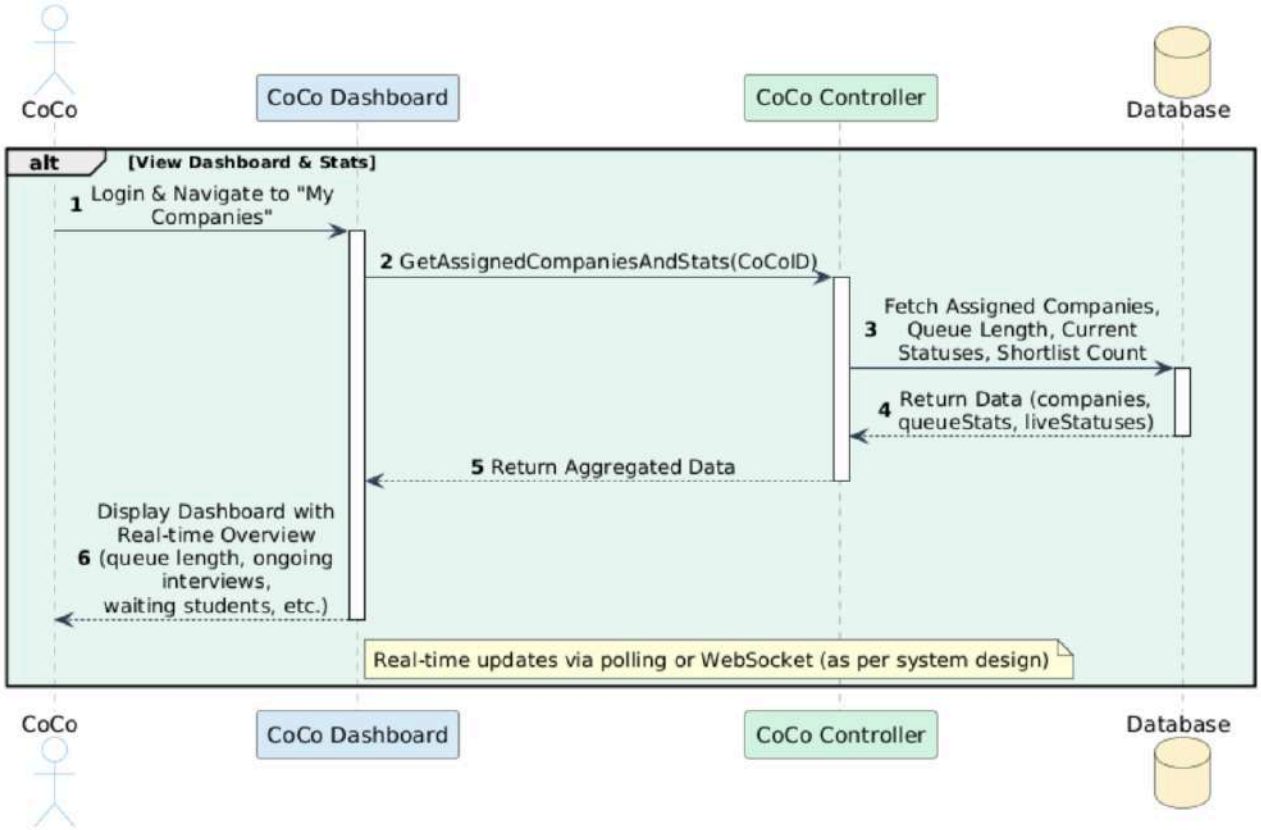


II. Admin1 : COCO (Company Coordinator)

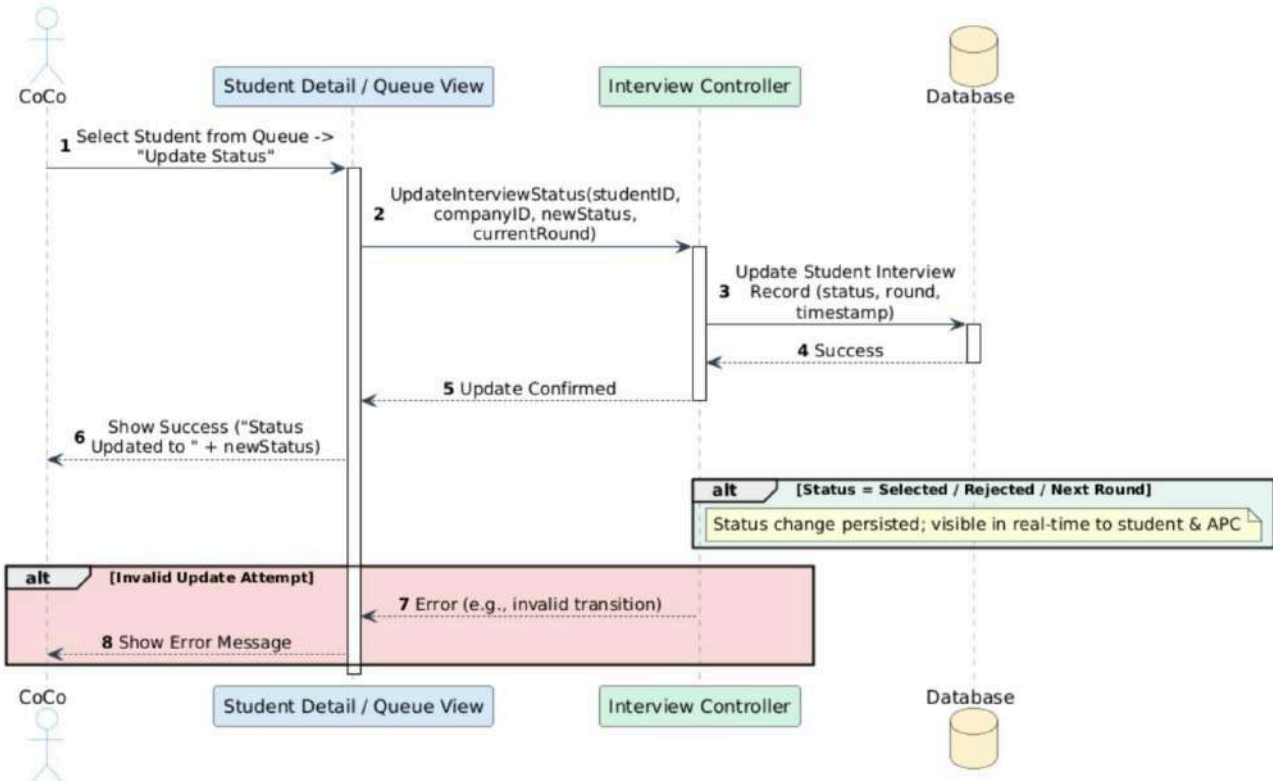
A. CoCo Login & Dashboard Access



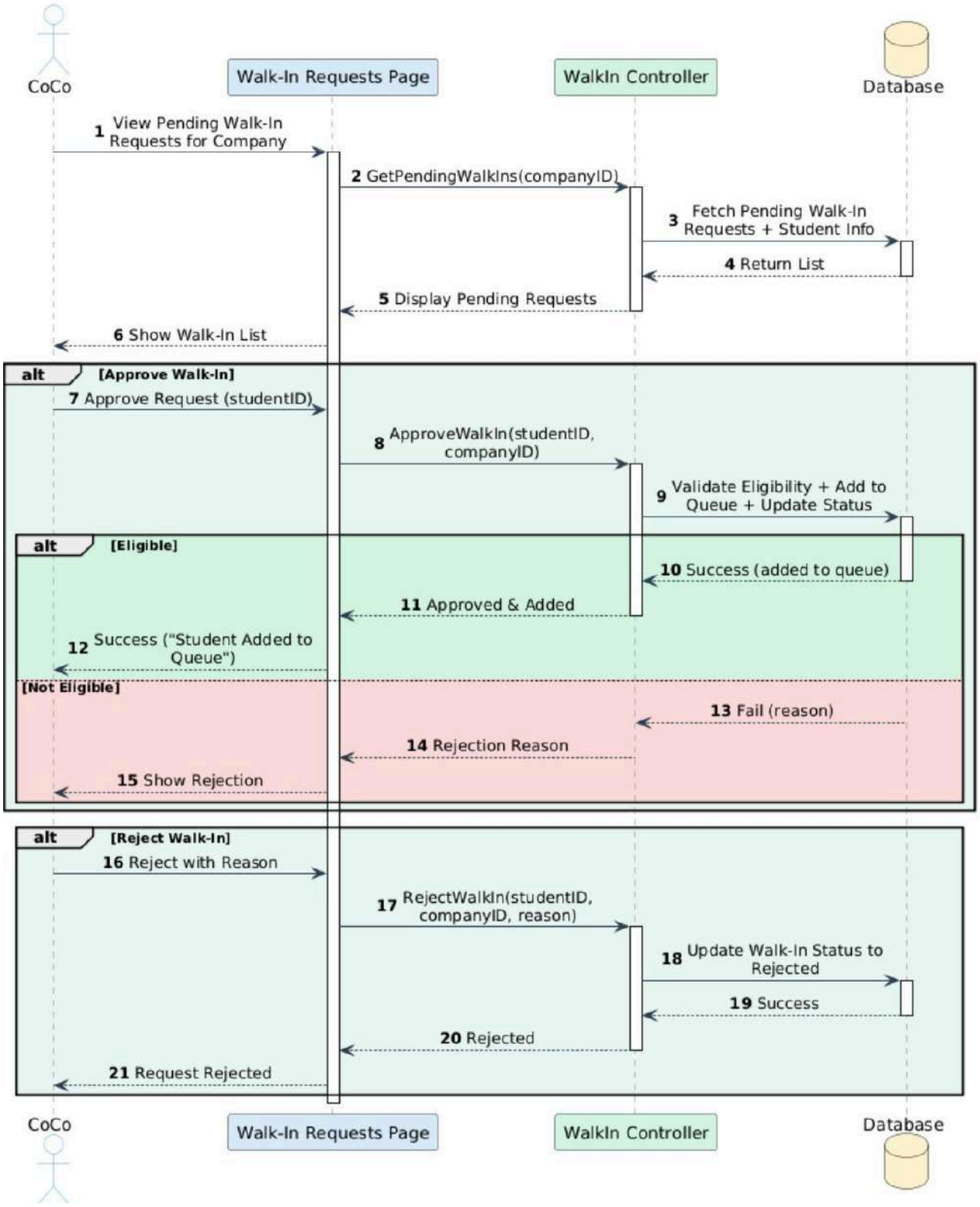
B. Manage Interview Queue



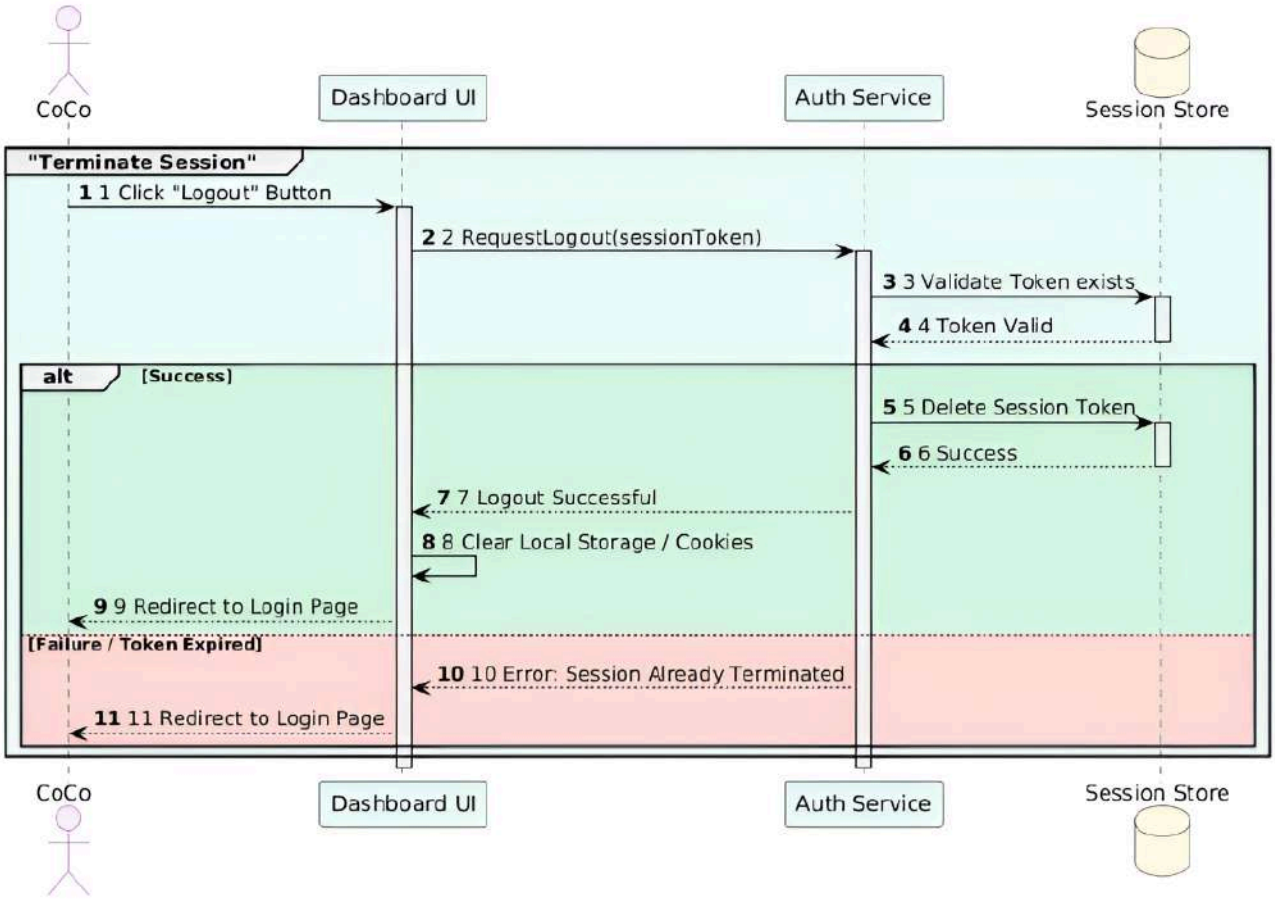
C. Update Student Interview Status / Round



D. Handle Walk-In Interview Request

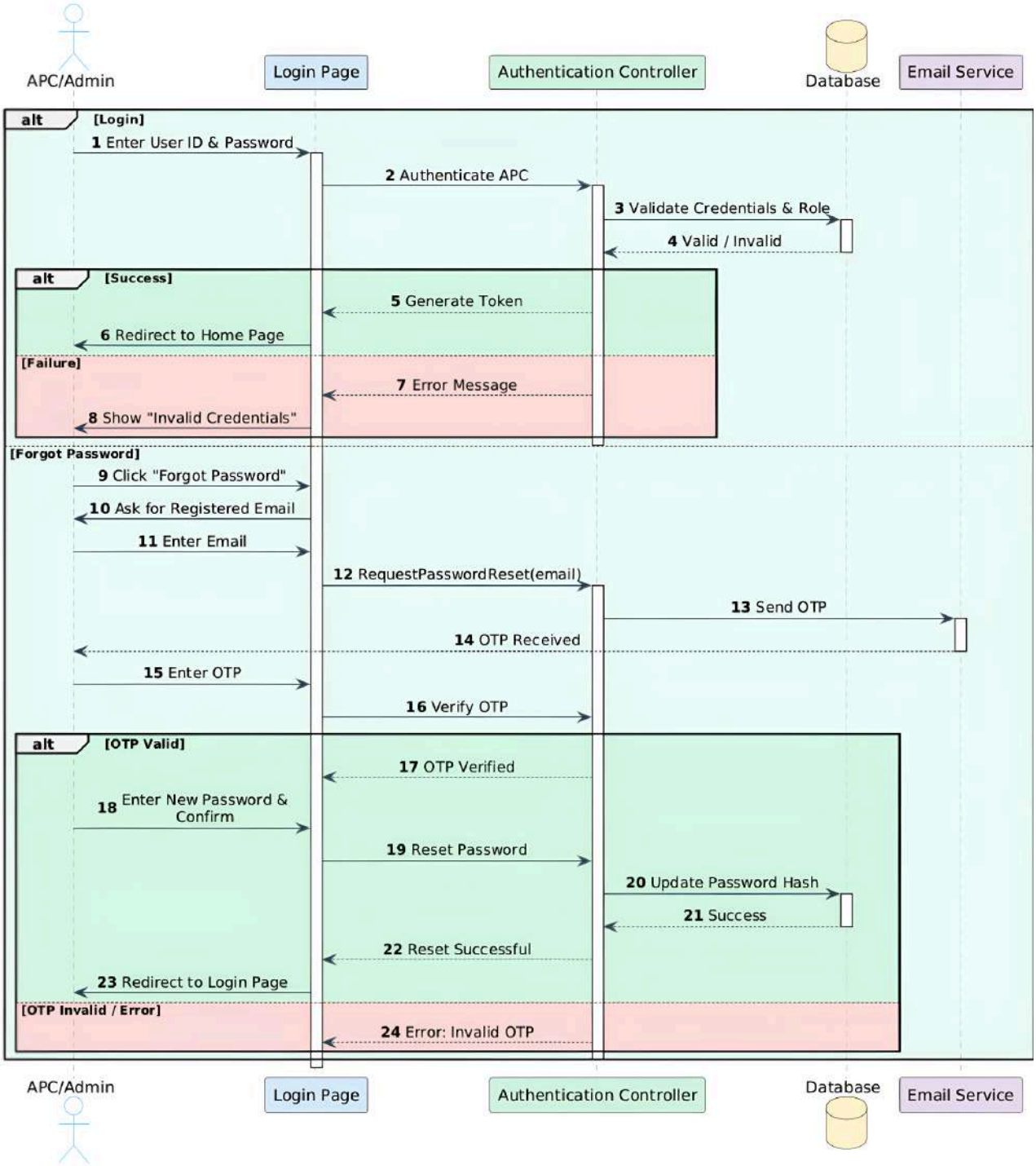


E. Logout

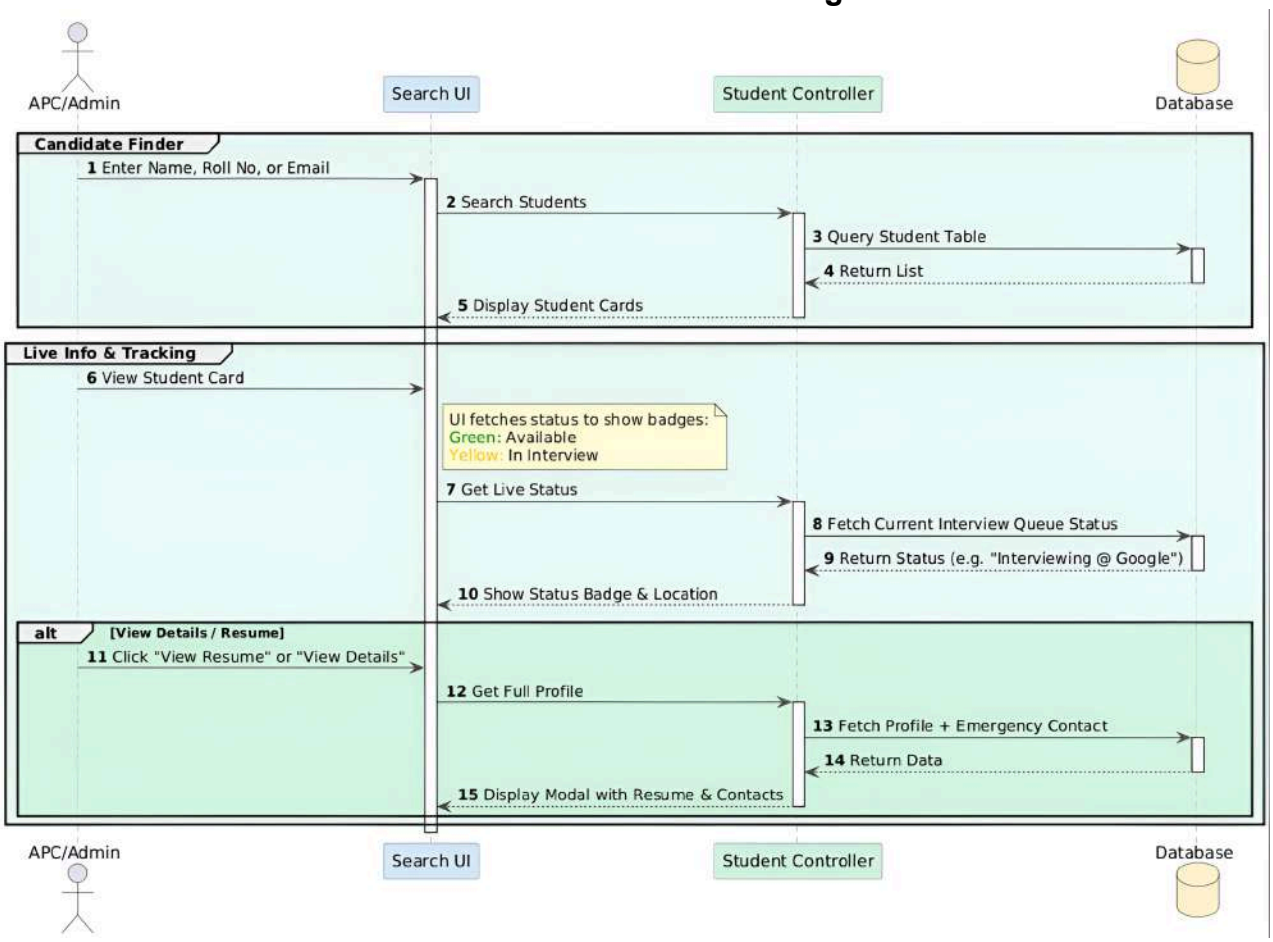

CoCo
Dashboard UI
Auth Service
Session Store
"Terminate Session"
1 1 Click "Logout" Button
2 2 RequestLogout(sessionToken)
3 3 Validate Token exists
4 4 Token Valid
alt
[Success]
5 5 Delete Session Token
6 6 Success
7 7 Logout Successful
8 8 Clear Local Storage / Cookies
9 9 Redirect to Login Page
[Failure / Token Expired]
10 10 Error: Session Already Terminated
11 11 Redirect to Login Page
CoCo
Dashboard UI
Auth Service
Session Store

III. Admin 2: APC / Admin

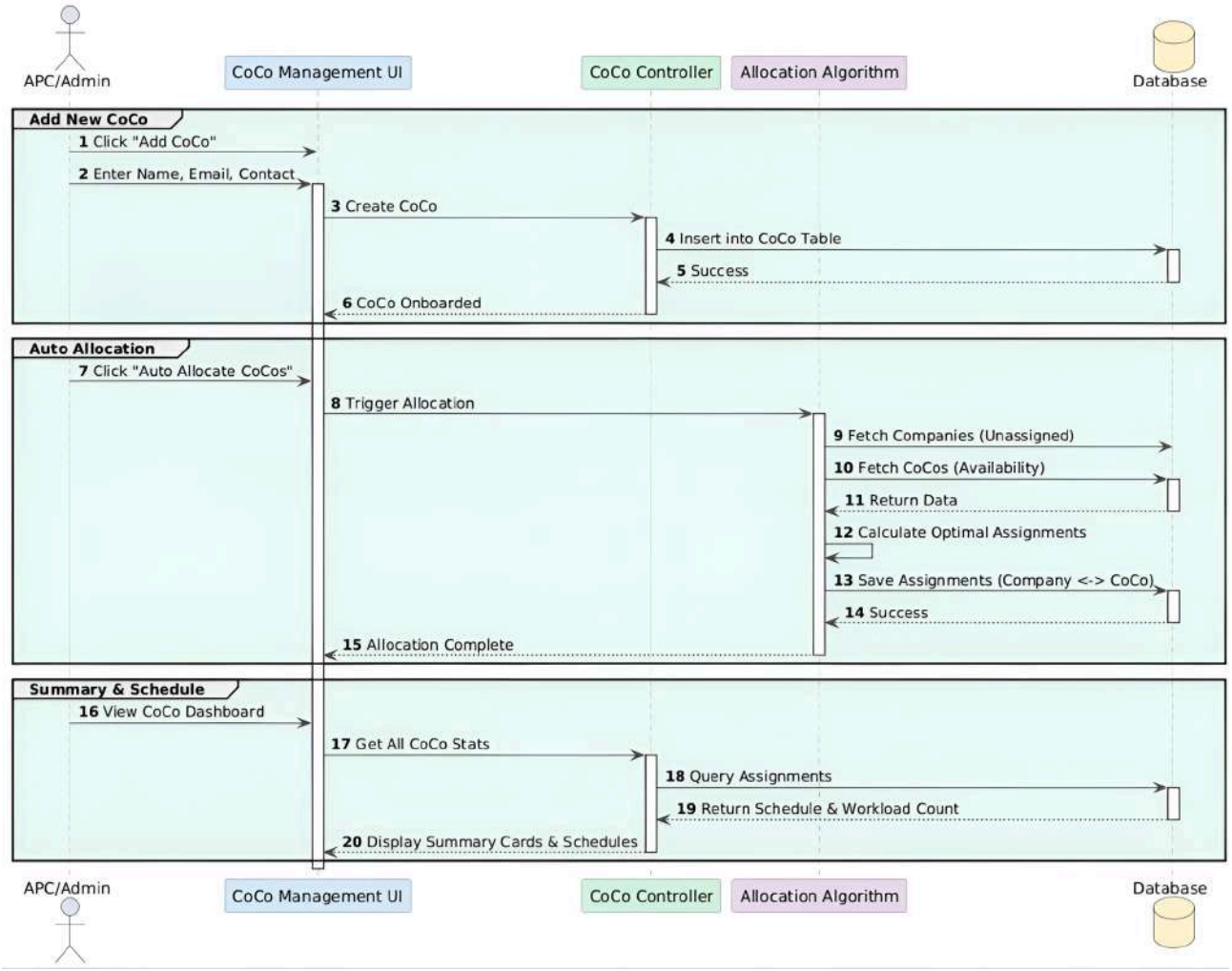
A. APC Login and Dashboard Access



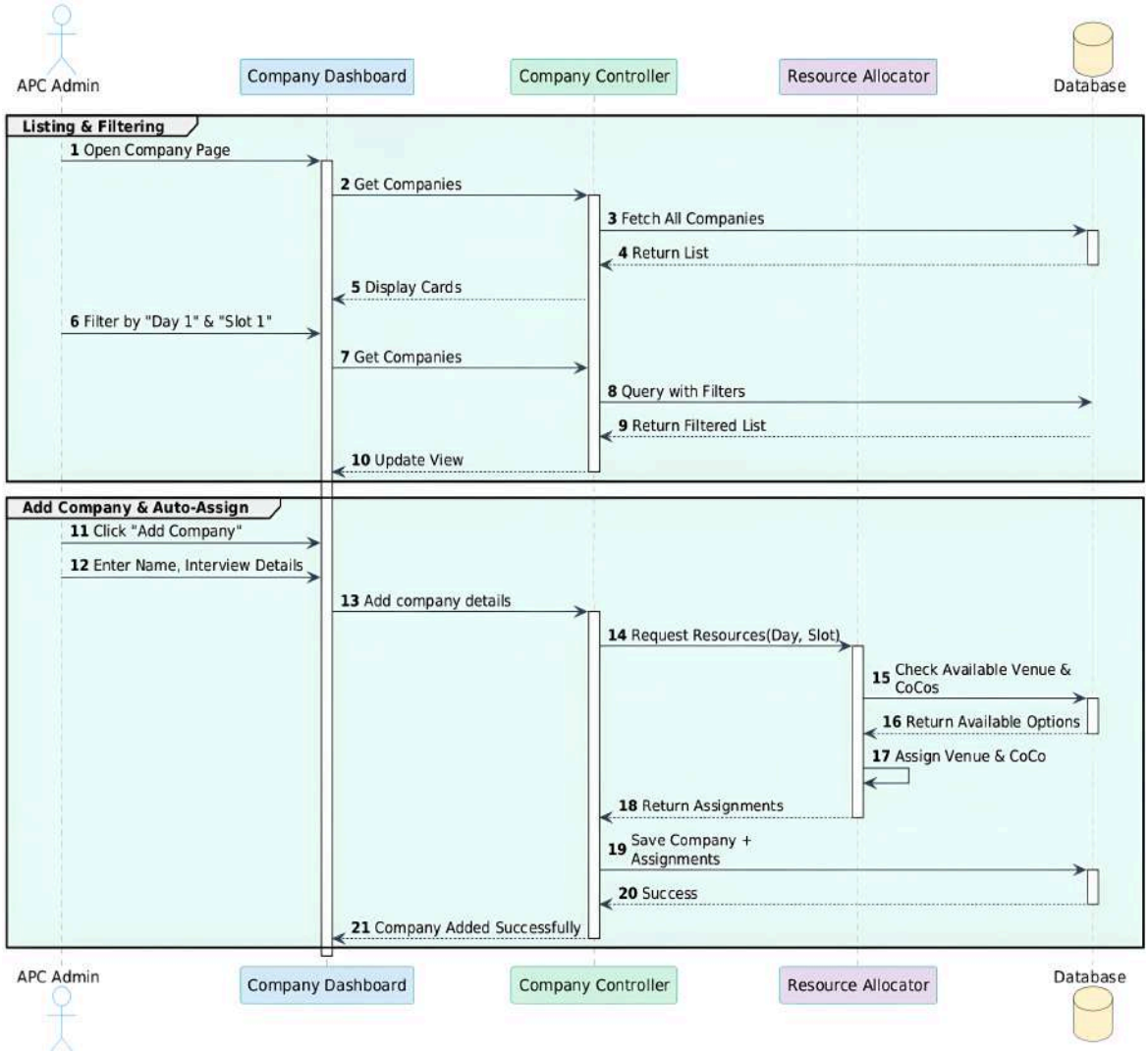
B. Global Student Search and Status Tracking



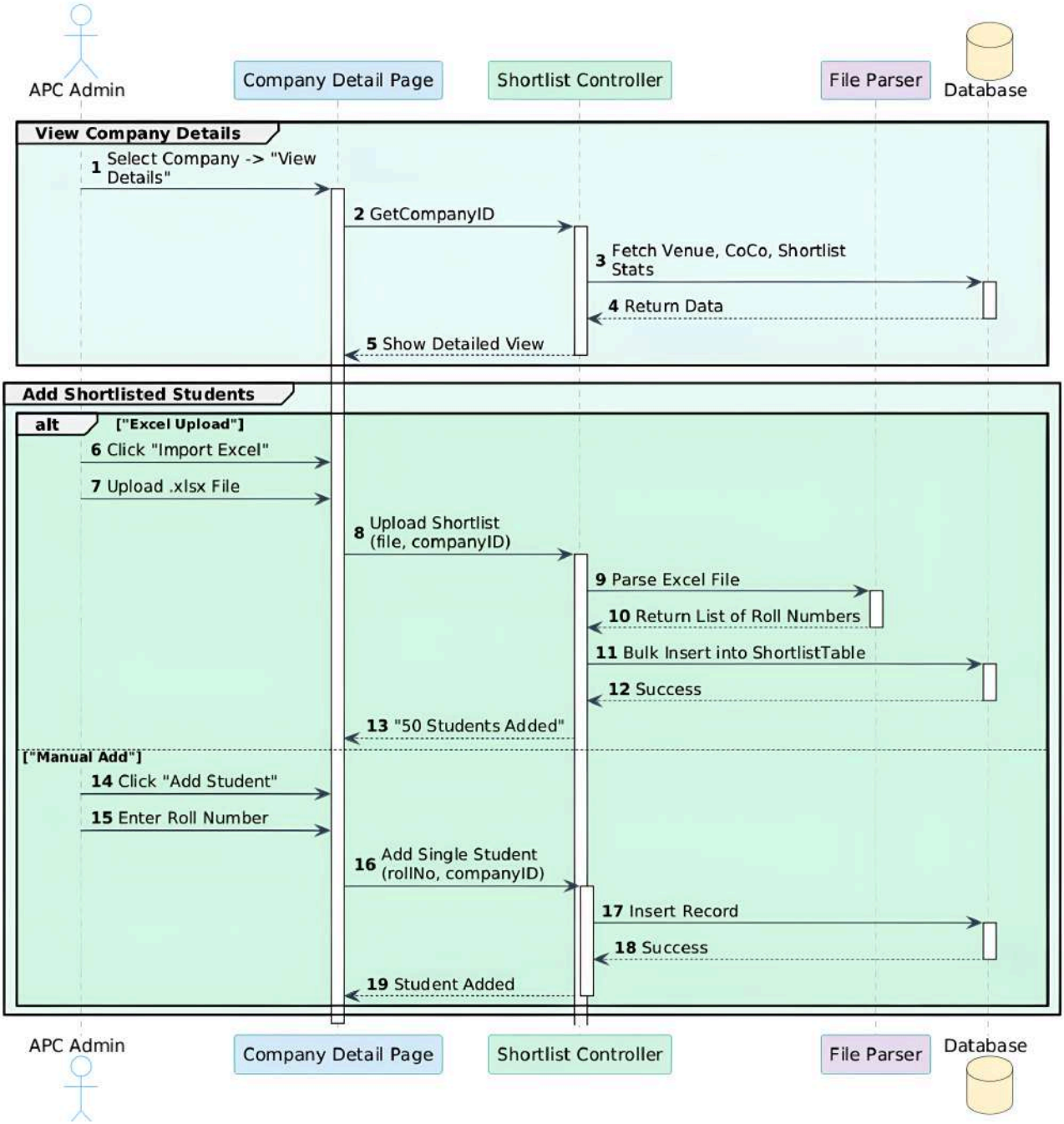
C. Manage and Onboard Company Coordinators (CoCos)



D. Manage Company and Slot Details

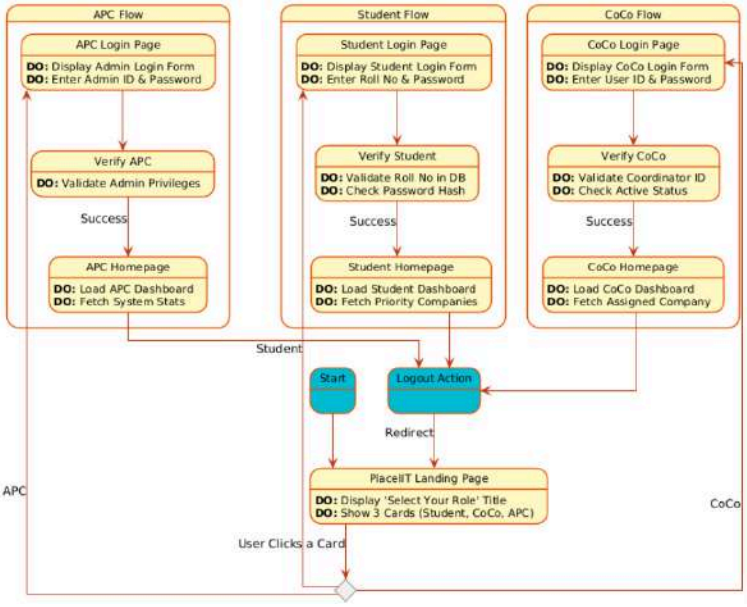


E. Add and Manage Student Shortlists

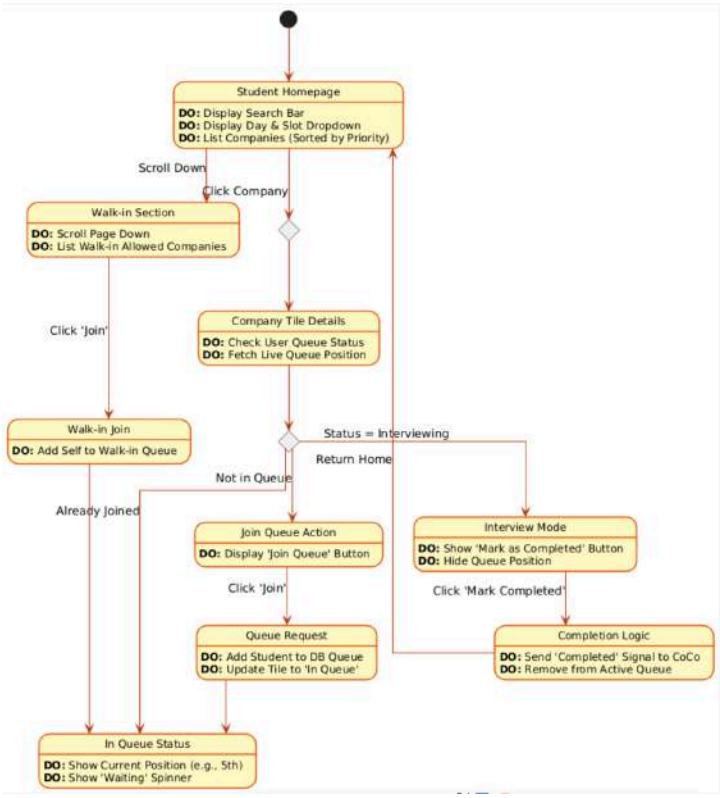


3.4 State Diagrams

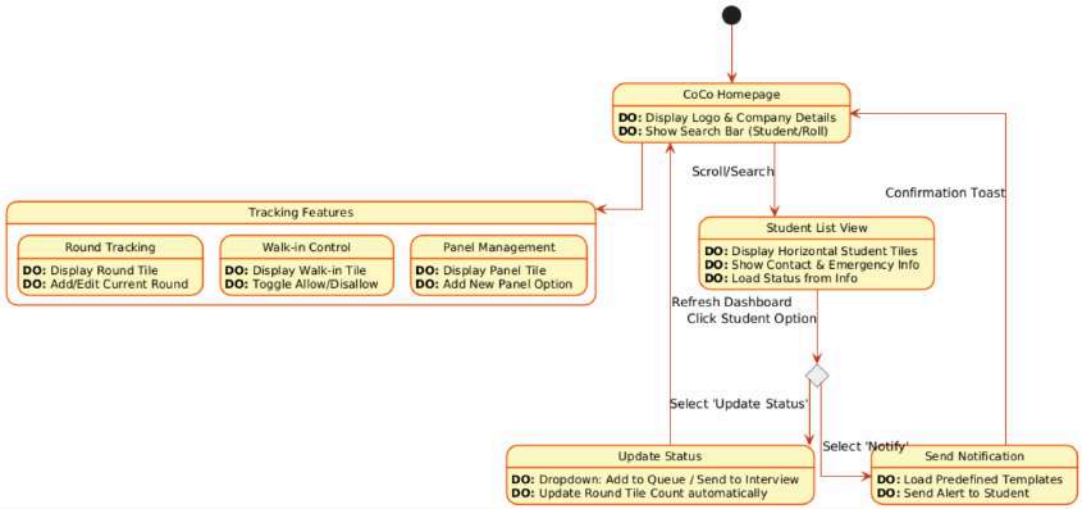
3.4.1 Login and Dashboard



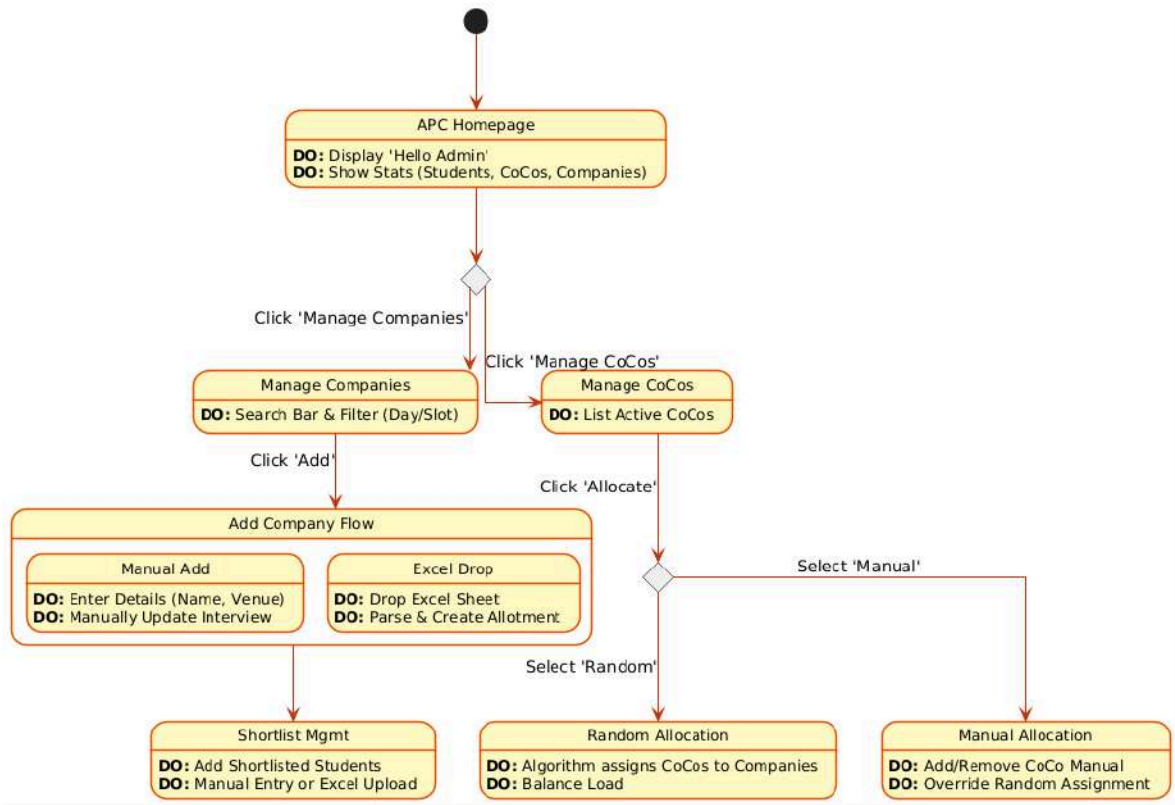
3.4.2 Student Dashboard



3.4.3 Co-Co Dashboard

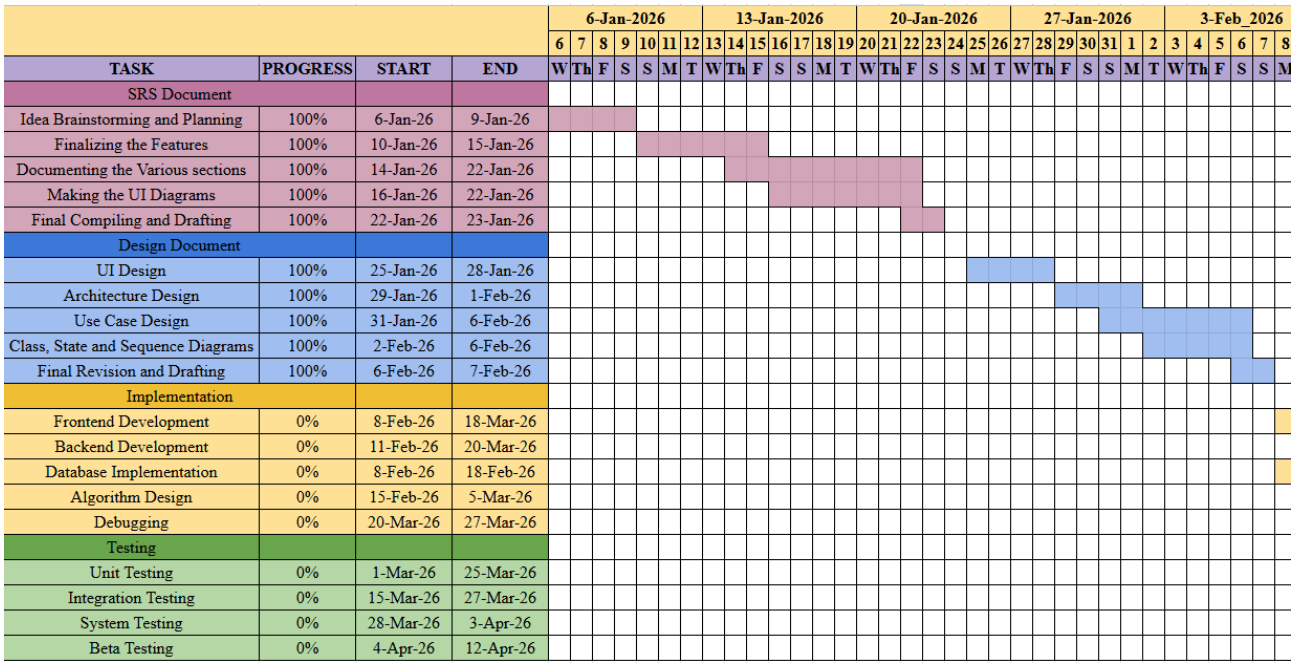


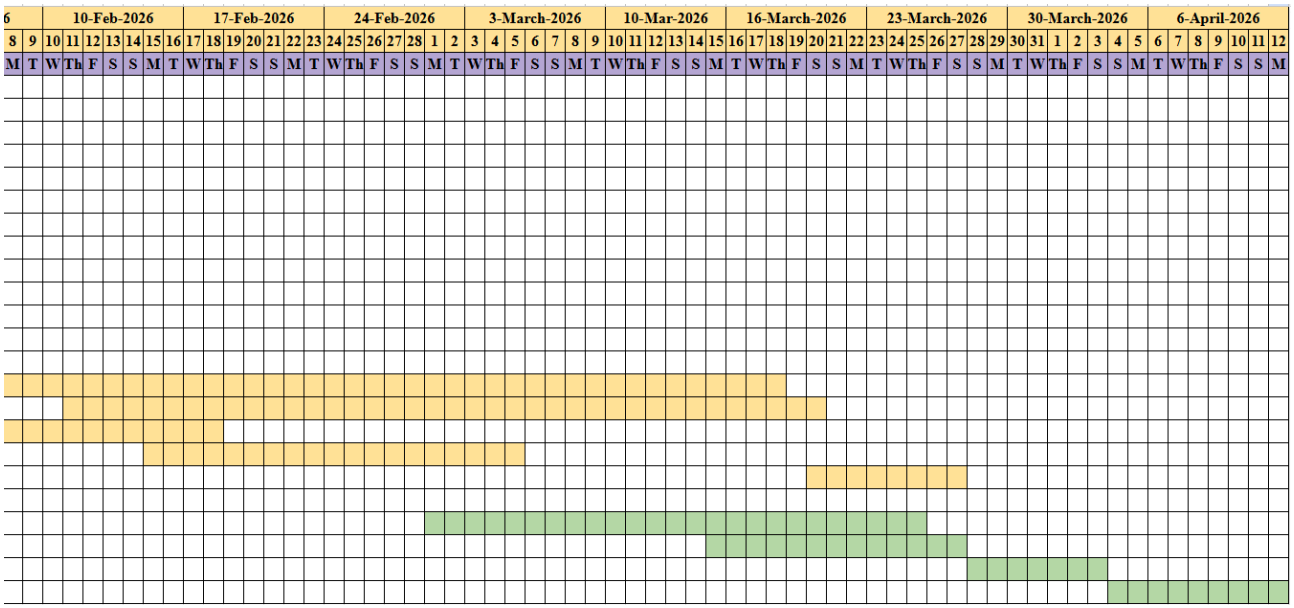
3.4.4 APC Dashboard



• Project Plan

TASK	PROGRESS	START	END
SRS Document			
Idea Brainstorming and Planning	100%	6-Jan-26	9-Jan-26
Finalizing the Features	100%	10-Jan-26	15-Jan-26
Documenting the Various sections	100%	14-Jan-26	22-Jan-26
Making the UI Diagrams	100%	16-Jan-26	22-Jan-26
Final Compiling and Drafting	100%	22-Jan-26	23-Jan-26
Design Document			
UI Design	100%	25-Jan-26	28-Jan-26
Architecture Design	100%	29-Jan-26	1-Feb-26
Use Case Design	100%	31-Jan-26	6-Feb-26
Class, State and Sequence Diagrams	100%	2-Feb-26	6-Feb-26
Final Revision and Drafting	100%	6-Feb-26	7-Feb-26
Implementation			
Frontend Development	0%	8-Feb-26	18-Mar-26
Backend Development	0%	11-Feb-26	20-Mar-26
Database Implementation	0%	8-Feb-26	18-Feb-26
Algorithm Design	0%	15-Feb-26	5-Mar-26
Debugging	0%	20-Mar-26	27-Mar-26
Testing			
Unit Testing	0%	1-Mar-26	25-Mar-26
Integration Testing	0%	15-Mar-26	27-Mar-26
System Testing	0%	28-Mar-26	3-Apr-26
Beta Testing	0%	4-Apr-26	12-Apr-26





Project Plan:

START DATE	END DATE	TASK DESCRIPTION	DURATION (days)	TEAM MEMBERS
		SRS Document		
6-Jan-26	9-Jan-26	Idea Brainstorming and Planning	4	Full Team
10-Jan-26	15-Jan-26	Finalizing the Features	6	Full Team
14-Jan-26	22-Jan-26	Documenting the Various sections	9	Abhirup, Anoop, Harsha, Krish, Devarsh, Kartik
16-Jan-26	22-Jan-26	Making the UI Diagrams	7	Pravesh, Joy, Samanvith, Sumath
22-Jan-26	23-Jan-26	Final Compiling and Drafting	2	Full Team
		Design Document		
25-Jan-26	28-Jan-26	UI Design	4	Samanvith, Pravesh
29-Jan-26	1-Feb-26	Architecture Design	4	Harsha, Krish
31-Jan-26	6-Feb-26	Use Case Design	7	Joy, Pravesh, Devarsh
2-Feb-26	6-Feb-26	Class, State, and Sequence Diagrams	5	Abhirup, Sumath, Kartik
6-Feb-26	7-Feb-26	Final Revision and Drafting	2	Full Team
		Implementation		

8-Feb-26	18-Mar-26	Frontend Development	39	Joy Vardhan, Samanvith
11-Feb-26	20-Mar-26	Backend Development	38	Kartik, Devarsh, Pravesh
8-Feb-26	18-Mar-26	Database Implementation	39	Harsha, Anoop, Krish
15-Feb-26	5-Mar-26	Algorithm Design	19	Abhirup, Sumath
20-Mar-26	27-Mar-26	Debugging	8	Krish, Pravesh, Kartik
		Testing		
1-Mar-26	25-Mar-26	Unit Testing	25	Full Team
15-Mar-26	27-Mar-26	Integration Testing	13	Full Team
28-Mar-26	3-Apr-26	System Testing and Preparing User Manual	4	Full Team
4-Apr-26	12-Apr-26	Code Review, Improvement and Beta Testing	9	Full Team
13-Apr-26	23-Apr-26	Final Project Review	11	Full team

Appendix A - Group Log

S.No	Date	Venue	Summary of the meeting
1	27 January 2026	Online	Discussed the UI design and architecture, divided into teams, and distributed the workload among the members.
2	30 January 2026	Cafè Coffee Day	Discussed the document's progress and finalized the various classes, designs, and overview of the algorithms to be used.
3	03 February 2026	Hall 2	Checked the progress of all members and clarified specific software design aspects. Finalized UI, state, and sequence diagrams and validated overall document progress.
4	05 February 2026	RM Ground Floor	Discussed document progress with the Teaching Assistant and resolved remaining doubts.
5	06 February 2026	Hall 2	Conducted final team meeting to consolidate work and prepare a robust final draft of the Software Design Document.